Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»
КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

«Разработка базы данных для хранения и обработки стоматологических снимков челюсти человека»

Студент **ИУ7-63Б** (Группа) _____ (Подпись, дата) **А. К. Пресняков** (И. О. Фамилия)

Руководитель _____ (Подпись, дата) **П. В. Дроздецкая** (И. О. Фамилия)

2026 г.

РЕФЕРАТ

Расчетно-пояснительная записка содержит сведения о проектировании и реализации базы данных для хранения и обработки стоматологических снимков челюсти человека.

БАЗА ДАННЫХ, PostgreSQL, СТОМАТОЛОГИЧЕСКИЙ СНИМОК, МЕДИЦИНСКАЯ КАРТА, РЕЛЯЦИОННАЯ МОДЕЛЬ, ДИАГНОЗ, Redis, C++, libpqxx.

Цель работы: проектирование и разработка информационной системы для хранения сведений о пациентах стоматологической клиники, медицинских картах, цифровых снимках челюсти, врачебных заключениях, клиниках, оборудовании и типах исследований.

В ходе работы выполняется анализ предметной области, формализация требований к базе данных и приложению доступа, проектирование реляционной схемы базы данных, разработка приложения доступа к данным, реализация внешнего кэширования с использованием Redis, проектирование ролевой модели на уровне базы данных, а также исследование влияния индексов на время выполнения запросов.

В качестве системы управления базами данных используется PostgreSQL. Приложение доступа к данным разрабатывается на языке C++ с использованием библиотеки libpqxx. Для проверки работоспособности бизнес-логики и слоя доступа к данным используется технологический консольный интерфейс, работающий с тем же компонентом бизнес-логики, что и будущий графический интерфейс.

СОДЕРЖАНИЕ

РЕФЕРАТ **3**

ВВЕДЕНИЕ **6**

1 Аналитическая часть **8**

1.1 Анализ предметной области 8

1.2 Анализ существующих решений 9

1.3 Формализация задачи 10

1.4 Формализация данных 11

1.5 Формализация пользователей 13

1.6 Анализ моделей баз данных 14

Вывод 15

2 Конструкторская часть **16**

2.1 Архитектура системы 16

2.1.1 Уровень L1: контекст системы 16

2.1.2 Уровень L2: контейнеры 17

2.1.3 Уровень L3: компоненты 18

2.1.4 Уровень L4: компоненты технологического интерфейса . 19

2.2 Диаграмма классов приложения 20

2.3 Проектирование базы данных 22

2.4 Описание сущностей базы данных 23

2.5 Ограничения целостности 25

2.6 Проверка соответствия базы данных третьей нормальной форме 26

2.7 Ролевая модель базы данных 27

2.8 Проектирование триггеров и функций базы данных 28

2.8.1 Триггер обновления даты изменения медицинской карты 28

2.8.2 Функция получения снимков пациента 30

2.9 Проектирование внешнего кэширования 31

Вывод 32

3 Технологическая часть 33
3.1 Выбор средств реализации 33
3.2 Структура программного комплекса 34
3.3 Конфигурирование приложения 34
3.4 Логирование 35
3.5 Физическая реализация базы данных 36
3.6 Инициализация базы данных 37
3.7 Ролевая модель базы данных 38
3.8 Функции и триггеры базы данных 38
3.9 Компонент доступа к данным 40
3.10 Компонент бизнес-логики 41
3.11 Redis-кэширование 41
3.12 Графический интерфейс 42
3.13 Сборка проекта 44
3.13.1 Запуск приложения 45
3.14 Тестирование приложения 45

4 Исследовательская часть 48
4.1 Цель исследования 48
4.1.1 Технические характеристики 48
4.2 Описание исходных данных 49
4.3 Описание исследуемых запросов 49
4.4 Методика проведения исследования 51
4.5 Результаты исследования 52
4.6 Оценка ускорения запросов 53

ЗАКЛЮЧЕНИЕ 56

ПРИЛОЖЕНИЕ А 59

ВВЕДЕНИЕ

Современная стоматология активно использует цифровые методы диагностики. Одним из основных источников диагностической информации являются рентгеновские, панорамные и томографические снимки челюсти пациента. Такие снимки используются врачами для выявления патологий, планирования лечения, контроля динамики состояния пациента и формирования медицинских заключений.

Рост объема цифровых медицинских данных приводит к необходимости централизованного хранения снимков и связанных с ними сведений: информации о пациенте, типе исследования, оборудовании, клинике, враче и поставленном диагнозе. Использование разрозненных файловых хранилищ и ручного учета усложняет поиск данных, повышает риск потери информации и затрудняет контроль доступа к медицинским сведениям.

Актуальность работы заключается в необходимости разработки специализированной базы данных и приложения доступа, ориентированных на хранение и обработку стоматологических снимков челюсти человека. Такая система должна обеспечивать целостность данных, поддержку связей между медицинскими сущностями, разграничение доступа пользователей и возможность повышения скорости получения часто используемой информации за счет внешнего кэширования.

Целью курсовой работы является проектирование и разработка базы данных для хранения и обработки стоматологических снимков челюсти человека, а также приложения доступа к этой базе данных.

Для достижения поставленной цели необходимо решить следующие задачи:

— проанализировать предметную область хранения и обработки стоматологических снимков;

— выполнить сравнительный анализ существующих решений;

— сформулировать требования к разрабатываемой базе данных и приложению доступа;

— формализовать данные, подлежащие хранению в базе данных;

— спроектировать реляционную схему базы данных и проверить ее соответствие третьей нормальной форме;

— спроектировать ролевую модель на уровне базы данных;

— реализовать базу данных и приложение доступа к данным;

— реализовать внешний механизм кэширования с использованием Redis;

— провести тестирование разработанного функционала;

— исследовать влияние применения индексов на время выполнения запросов к базе данных.

Объектом разработки является информационная система стоматологической клиники, обеспечивающая хранение и обработку цифровых снимков челюсти человека. Предметом разработки является реляционная база данных и приложение доступа к ней.

1 Аналитическая часть

В данном разделе выполняется анализ предметной области хранения и обработки стоматологических снимков челюсти человека. Рассматриваются существующие программные решения, формализуются требования к разрабатываемой системе, выделяются основные категории хранимых данных и группы пользователей. Также проводится анализ моделей баз данных и обосновывается выбор реляционной модели данных.

1.1 Анализ предметной области

Стоматологическая диагностика включает сбор, хранение и анализ информации о состоянии зубочелюстной системы пациента. Одним из основных источников диагностической информации являются цифровые стоматологические снимки: панорамные снимки, прицельные снимки, телерентгенограммы и результаты компьютерной томографии. Такие изображения используются врачом для выявления патологий, планирования лечения, контроля динамики состояния пациента и документирования результатов медицинского обследования.

При работе со стоматологическими снимками необходимо хранить не только само изображение, но и набор связанных с ним данных: сведения о пациенте, дате загрузки снимка, типе исследования, оборудовании, на котором был получен снимок, медицинском специалисте, выполнившем анализ, и заключении врача. Такой подход соответствует общей практике хранения медицинских изображений, где важную роль играют как бинарные данные изображения, так и метаданные исследования [1].

В рамках рассматриваемой предметной области участвуют следующие основные категории пользователей:

— врач-стоматолог — анализирует снимки пациентов, формирует заключения и просматривает историю обследований;

— рентгенолог или лаборант — загружает новые снимки, указывает тип исследования и оборудование;

— администратор — управляет справочниками, клиниками, оборудованием и правами доступа.

Основная задача разрабатываемой системы заключается в централизованном хранении стоматологических снимков и связанных с ними медицинских данных. Система должна обеспечивать быстрый поиск снимков пациента, добавление новых исследований, создание врачебных заключений и управление справочной информацией.

1.2 Анализ существующих решений

В области хранения и обработки медицинских изображений существуют как универсальные, так и специализированные программные решения. К универсальным решениям относятся DICOM-серверы и PACS-системы, предназначенные для хранения медицинских изображений различных видов. К специализированным решениям относятся программные комплексы производителей стоматологического оборудования, ориентированные на работу с конкретными аппаратами и форматами данных.

В качестве примера специализированного решения можно выделить программное обеспечение Planmeca Romexis, предназначенное для работы со стоматологическими изображениями и данными пациентов [2]. Примером открытого решения для хранения медицинских изображений является Orthanc, представляющий собой легковесный DICOM-сервер [3].

Для сравнения существующих решений были выделены следующие критерии:

— специализация на стоматологических снимках;
— возможность централизованного хранения изображений;
— гибкость ролевой модели доступа;
— возможность интеграции с собственной бизнес-логикой;
— стоимость внедрения и сопровождения;
— возможность использования внешнего кэширования.

Сравнение существующих решений приведено в таблице 1.1.

Таблица 1.1 — Сравнение существующих решений

Критерий	Planmeca Romexis	Orthanc	DentoBase
Специализация на стоматологии	да	нет	да
Централизованное хранение снимков	да	да	да
Гибкая ролевая модель	ограничена настройками продукта	требует настройки	проектируется на уровне БД
Возможность изменения бизнес-логики	ограничена	возможна через расширения	полная, так как система разрабатывается самостоятельно
Стоимость внедрения	высокая	низкая	низкая
Внешнее кэширование	зависит от конфигурации	требует отдельной настройки	проектируется как часть системы

Из таблицы видно, что существующие решения либо обладают избыточной функциональностью и высокой стоимостью, либо требуют дополнительной настройки для применения в стоматологической предметной области. Разрабатываемая система ориентирована на хранение и обработку именно стоматологических снимков и может быть адаптирована под требования конкретной клиники.

1.3 Формализация задачи

Необходимо разработать базу данных и приложение доступа к ней для хранения и обработки стоматологических снимков челюсти человека. Система должна обеспечивать хранение сведений о пациентах, врачах, лаборантах, клиниках, оборудовании, типах снимков, медицинских картах, снимках и врачебных заключениях.

В рамках поставленной цели необходимо решить следующие задачи:

— проанализировать предметную область;
— определить основные сущности и связи между ними;
— сформулировать требования к базе данных и приложению;
— спроектировать реляционную базу данных;
— проверить соответствие базы данных третьей нормальной форме;
— спроектировать ролевую модель на уровне базы данных;
— спроектировать функции и триггеры на стороне базы данных;

— реализовать приложение доступа к базе данных;

— реализовать механизм внешнего кэширования с использованием Redis;

— провести тестирование разработанного приложения;

— исследовать влияние индексов на время выполнения запросов.

Разрабатываемая система должна поддерживать следующие основные варианты использования:

— регистрация пациента;

— поиск пациента;

— создание медицинской карты;

— добавление нового стоматологического снимка;

— просмотр снимка;

— добавление врачебного заключения;

— управление справочниками;

— управление врачами, клиниками и оборудованием.

1.4 Формализация данных

На основании анализа предметной области были выделены следующие категории данных:

— пациент;

— врач;

— клиника;

— лаборант;

— оборудование;

— тип снимка;

— стоматологический снимок;

— медицинская карта;

— диагноз или врачебное заключение;

— связь врача и клиники.

Сведения о категориях данных приведены в таблице 1.2.

Таблица 1.2 — Категории и сведения о данных

Категория	Сведения
Пациент	идентификатор, ФИО, дата рождения
Врач	идентификатор, специальность, список клиник, в которых работает врач
Клиника	идентификатор, адрес
Лаборант	идентификатор, ФИО, клиника работы
Оборудование	идентификатор, модель, клиника размещения
Тип снимка	идентификатор, название типа исследования
Снимок	идентификатор, бинарные данные изображения, дата загрузки, пациент, лаборант, оборудование, тип снимка
Медицинская карта	идентификатор, пациент, дата создания, дата обновления, признак активности, примечания
Диагноз	идентификатор, текст заключения, дата создания, снимок, врач, медицинская карта
Связь врача и клиники	идентификатор врача, идентификатор клиники

Между сущностями предметной области существуют следующие связи:

— один пациент может иметь несколько снимков;

— один пациент может иметь несколько медицинских карт;

— одна медицинская карта может содержать несколько заключений;

— один снимок может иметь одно заключение;

— один врач может создать несколько заключений;

— одна клиника может иметь несколько единиц оборудования;

— одна клиника может иметь несколько лаборантов;

— один тип снимка может использоваться для нескольких снимков;

— один врач может работать в нескольких клиниках, и одна клиника может иметь нескольких врачей.

Последняя связь является связью многие-ко-многим. В дальнейшем она будет реализована с помощью ассоциативного отношения.

ER-диаграмма сущностей предметной области приведена на рисунке 1.1.

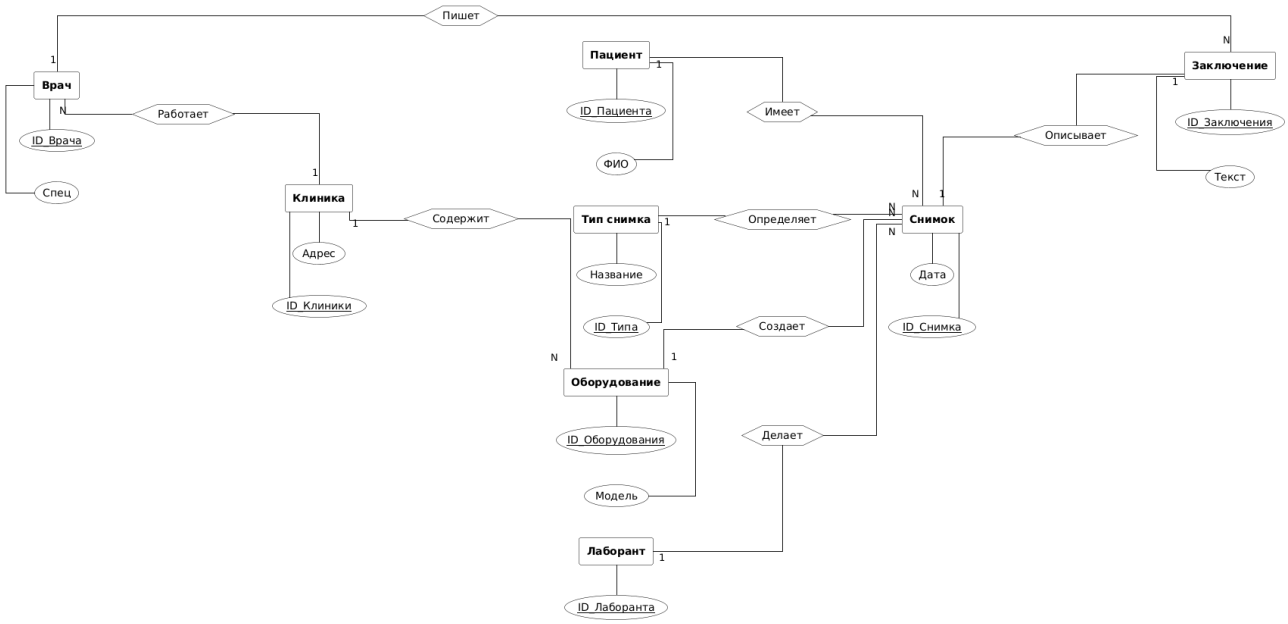


Рисунок 1.1 — ER-диаграмма сущностей предметной области

1.5 Формализация пользователей

В системе выделяются следующие группы пользователей:

— врач-стоматолог;

— рентгенолог или лаборант;

— администратор.

Врач-стоматолог просматривает данные пациентов, изучает снимки и формирует медицинские заключения. Рентгенолог загружает снимки и указывает сведения об исследовании. Администратор управляет справочниками, клиниками, оборудованием, пользователями и правами доступа.

Описание ролей пользователей приведено в таблице 1.3.

Таблица 1.3 — Пользователи системы

Пользователь	Действия
Врач-стоматолог	поиск пациента, просмотр снимков, просмотр медицинской карты, добавление заключения
Рентгенолог	регистрация пациента, загрузка снимка, выбор типа снимка и оборудования
Администратор	управление справочниками, клиниками, врачами, оборудованием и правами доступа

Диаграмма вариантов использования приведена на рисунке 1.2.

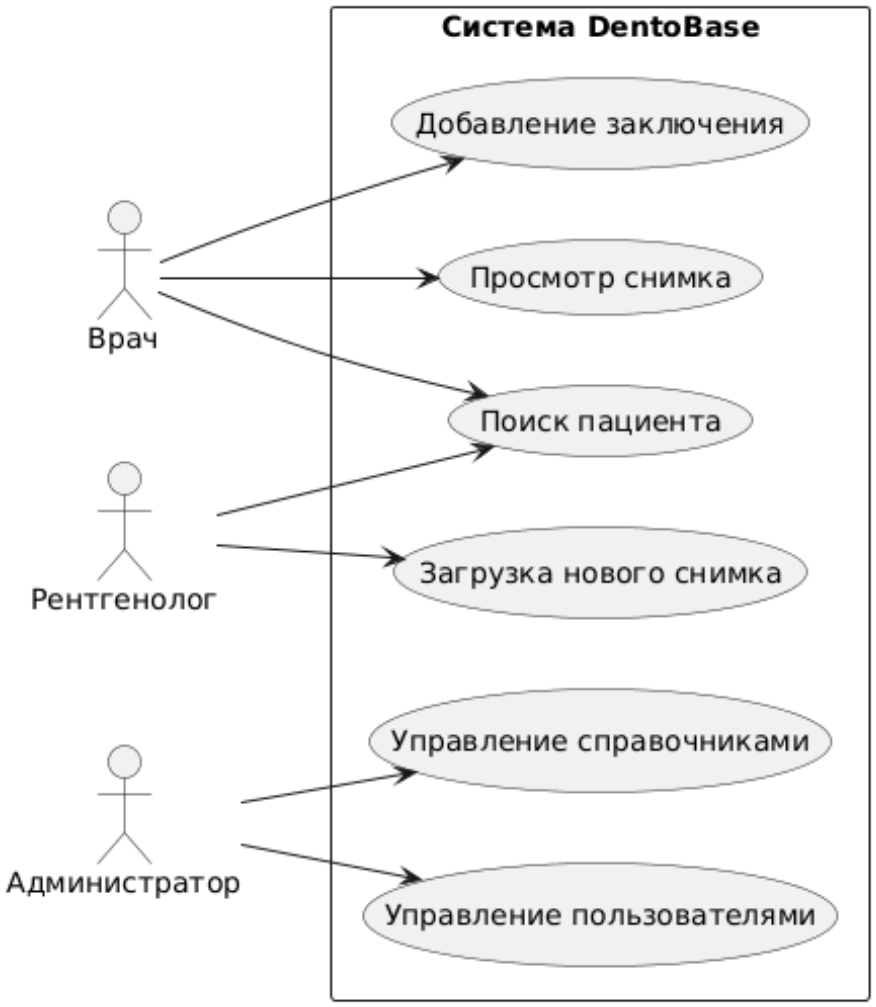


Рисунок 1.2 — Диаграмма вариантов использования

1.6 Анализ моделей баз данных

База данных представляет собой организованную совокупность данных, предназначенную для долговременного хранения и использования прикладными программами [4]. Система управления базами данных обеспечивает создание, изменение, поиск, защиту и поддержание целостности данных [5].

По модели организации данных можно выделить следующие основные типы баз данных:

— иерархические;

— сетевые;

— реляционные;

— объектно-реляционные;

— документоориентированные.

Иерархическая модель представляет данные в виде дерева. Такая модель

удобна для описания строго иерархичных структур, однако плохо подходит для предметной области, где между сущностями существует большое количество связей разных типов.

Сетевая модель позволяет описывать более сложные связи между данными, чем иерархическая, однако требует более сложной навигации между записями и теснее связывает прикладную логику со структурой хранения данных.

Документоориентированная модель удобна для хранения слабоструктурированных данных, но в рассматриваемой задаче важную роль играют строгие связи между пациентами, снимками, врачами, клиниками и медицинскими картами. Для таких данных требуется контроль ссылочной целостности.

Реляционная модель представляет данные в виде отношений, то есть таблиц, и позволяет задавать первичные ключи, внешние ключи и ограничения целостности [4, 8]. В рассматриваемой системе данные имеют четкую структуру, а связи между сущностями заранее известны. Поэтому реляционная модель является наиболее подходящей.

Для реализации базы данных выбрана PostgreSQL. Данная СУБД поддерживает реляционную модель данных, транзакции, индексы, ограничения целостности, роли, функции и триггеры [6, 9, 10]. Эти возможности необходимы для выполнения требований к разрабатываемой системе.

Вывод

В аналитической части была рассмотрена предметная область хранения и обработки стоматологических снимков челюсти человека. Были проанализированы существующие решения, сформулирована задача, выделены основные категории данных и пользователи системы. Также были рассмотрены модели баз данных и обоснован выбор реляционной модели данных и PostgreSQL в качестве СУБД.

2 Конструкторская часть

В данном разделе выполняется проектирование базы данных и архитектуры программного обеспечения для хранения и обработки стоматологических снимков челюсти человека. Описываются сущности базы данных, связи между ними, ограничения целостности, ролевая модель, схемы алгоритмов проектируемых триггеров и функций, а также архитектурные диаграммы системы.

2.1 Архитектура системы

Для описания архитектуры программного комплекса используется модель C4. Она позволяет рассматривать систему на нескольких уровнях детализации: от общего контекста до отдельных компонентов и классов.

2.1.1 Уровень L1: контекст системы

На уровне контекста разрабатываемая система рассматривается как единый программный комплекс, с которым взаимодействуют пользователи: врач-стоматолог, рентгенолог и администратор. Система использует PostgreSQL для долговременного хранения данных и Redis для внешнего кэширования часто запрашиваемой информации.

Контекстная диаграмма системы приведена на рисунке 2.1.

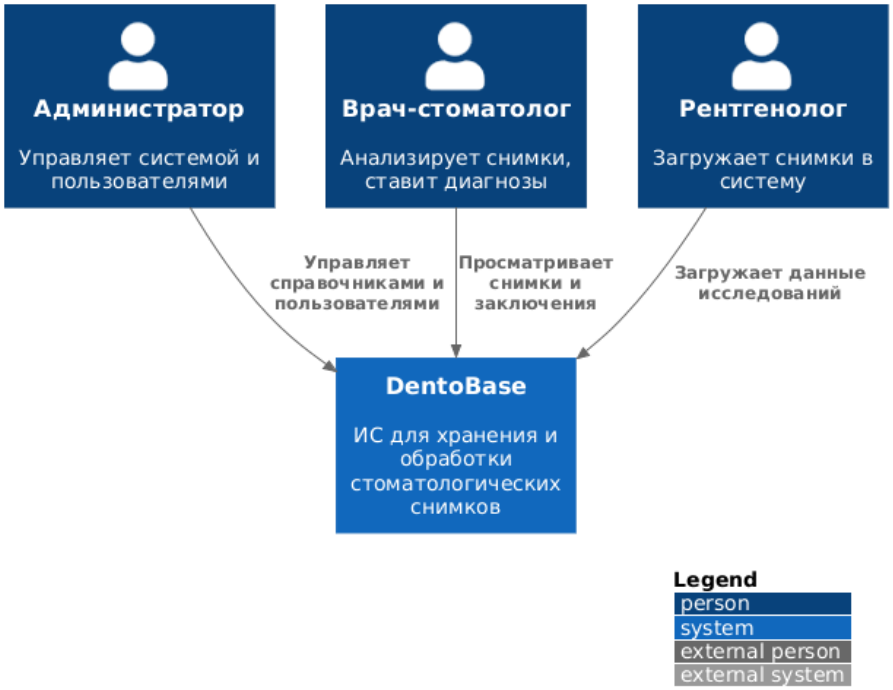


Рисунок 2.1 — C4 L1 — контекст системы

2.1.2 Уровень L2: контейнеры

На уровне контейнеров система разделяется на технологический пользовательский интерфейс, компонент бизнес-логики, базу данных PostgreSQL и InMemory-хранилище Redis. Технологический пользовательский интерфейс используется для системного тестирования вариантов использования и работает с тем же компонентом бизнес-логики, с которым в дальнейшем будет работать полноценный графический интерфейс.

Диаграмма контейнеров приведена на рисунке 2.2.

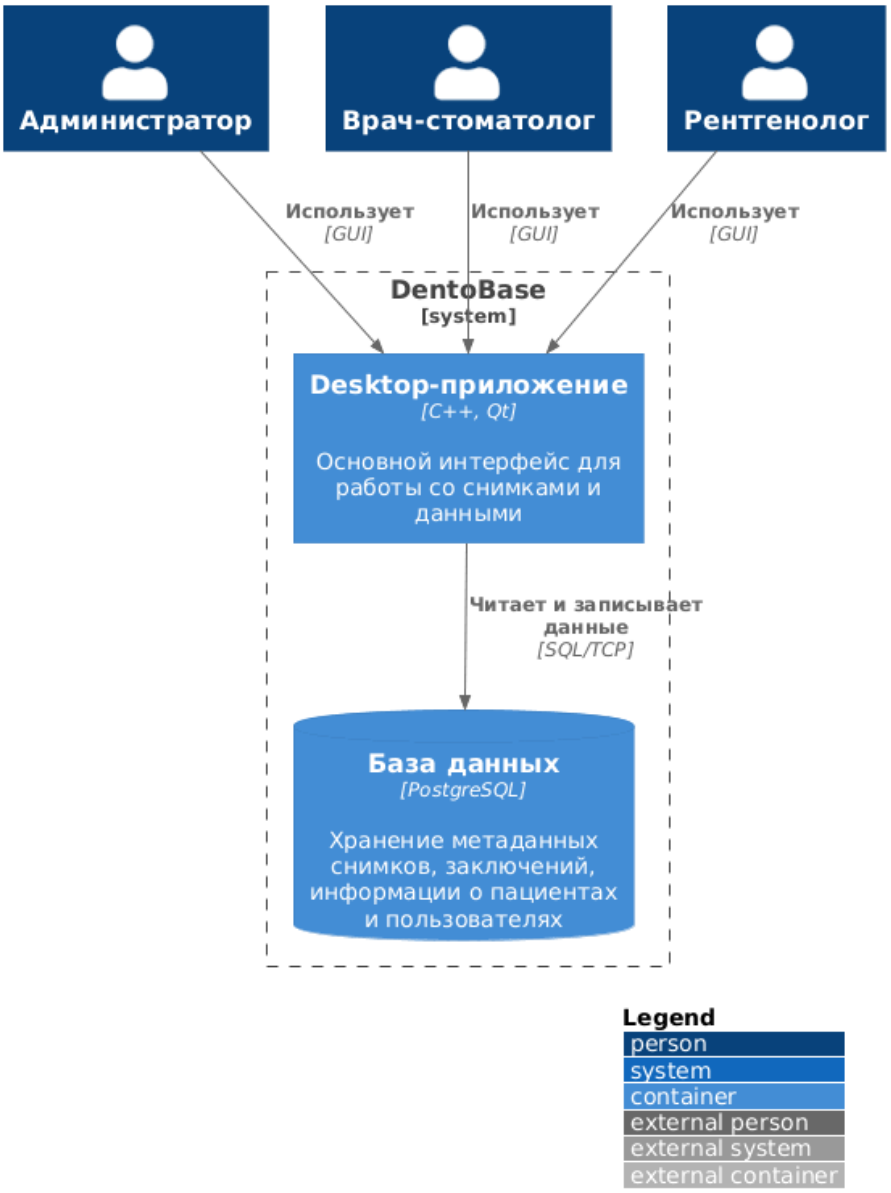


Рисунок 2.2 — C4 L2 — диаграмма контейнеров

2.1.3 Уровень L3: компоненты

На уровне компонентов выделяются пользовательский интерфейс, сервисы бизнес-логики, интерфейсы репозиториев, PostgreSQL-реализации репозиториев, SQL-builder-ы и компонент кэширования. Такое разделение позволяет отделить бизнес-логику от деталей хранения данных и упростить замену пользовательского интерфейса.

Диаграмма компонентов приведена на рисунке 2.3.

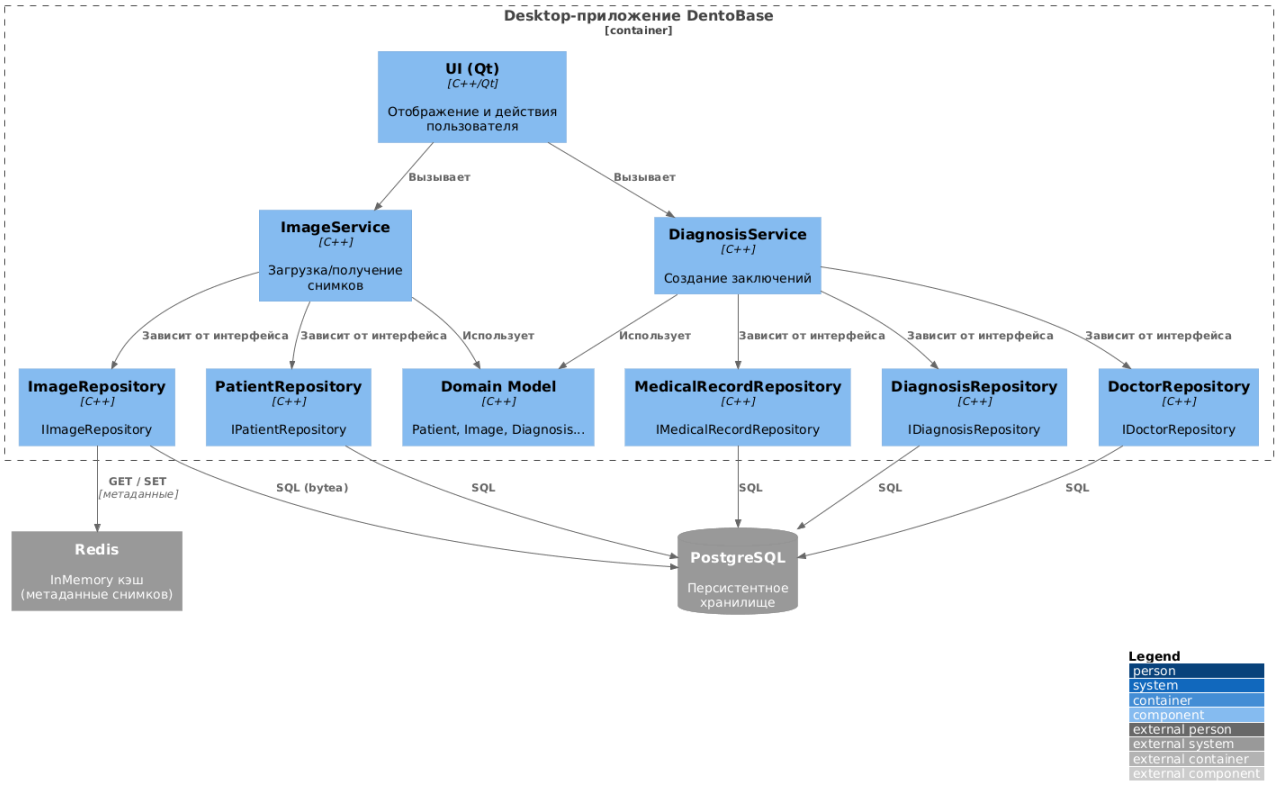


Рисунок 2.3 — C4 L3 — диаграмма компонентов

2.1.4 Уровень L4: компоненты технологического интерфейса

Технологический пользовательский интерфейс реализуется в виде консольного приложения. Он предназначен для проверки всех основных вариантов использования системы без разработки полноценного графического интерфейса. При этом технологический интерфейс не обращается к базе данных напрямую, а использует интерфейс прикладного сервиса. Это позволяет соблюдать принцип инверсии зависимостей и переиспользовать бизнес-логику в будущем графическом интерфейсе.

Диаграмма компонентов технологического интерфейса приведена на рисунке 2.4.

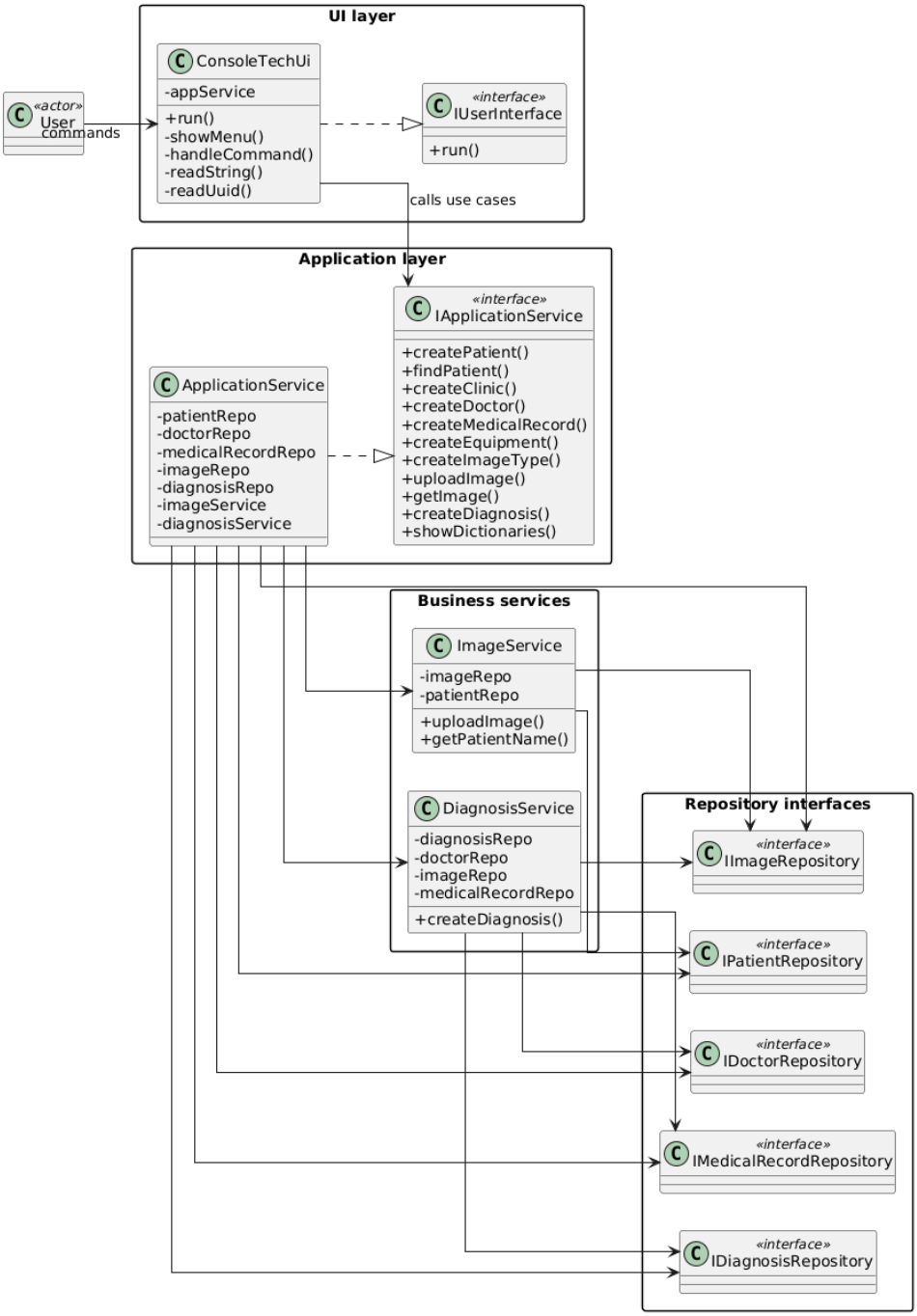


Рисунок 2.4 — C4 L4 — компоненты технологического пользовательского интерфейса

2.2 Диаграмма классов приложения

Приложение строится с разделением на доменный слой, слой репозиториев, слой бизнес-сервисов и слой пользовательского интерфейса. Доменный слой содержит структуры предметной области. Репозитории отвечают за доступ к данным. Сервисы реализуют бизнес-операции, а пользовательский интерфейс инициирует сценарии работы пользователя.

Полная диаграмма классов приложения приведена на рисунке 2.5.

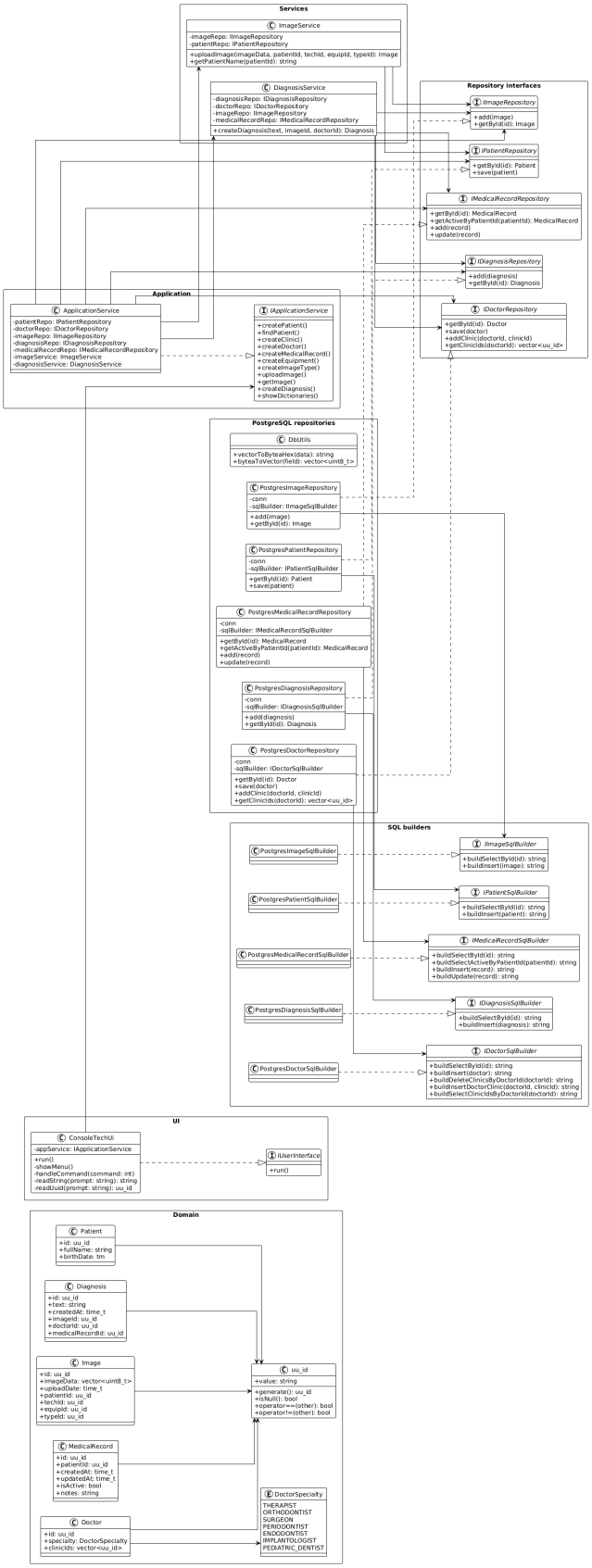


Рисунок 2.5 — Диаграмма классов приложения

В архитектуре приложения используются следующие принципы:

— зависимость бизнес-логики от абстракций репозиториев, а не от конкретной реализации PostgreSQL;

— разделение доменных моделей, сервисов и слоя доступа к данным;

— возможность замены технологического интерфейса на графический интерфейс без изменения бизнес-логики;

— инкапсуляция SQL-запросов в отдельных SQL-builder-ах.

2.3 Проектирование базы данных

Проектируемая база данных предназначена для хранения сведений о пациентах, врачах, клиниках, лаборантах, оборудовании, типах снимков, медицинских картах, стоматологических снимках и врачебных заключениях.

Диаграмма проектируемой базы данных приведена на рисунке 2.6.

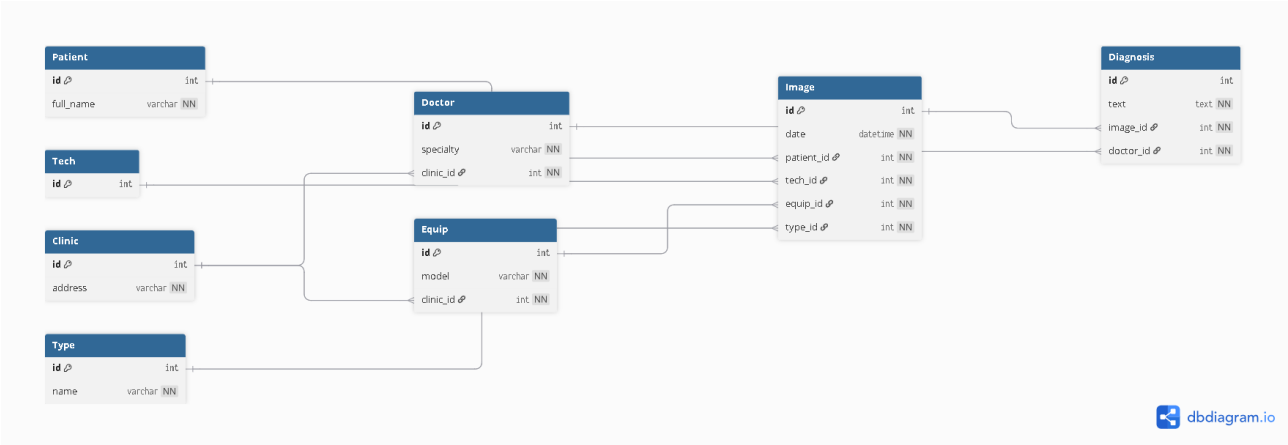


Рисунок 2.6 — Диаграмма проектируемой базы данных

В базе данных выделены следующие таблицы:

— `patients` — пациенты;

— `clinics` — стоматологические клиники;

— `doctors` — врачи;

— `doctor_clinics` — ассоциативная таблица связи врачей и клиник;

— `technicians` — лаборанты;

— `equipment` — оборудование;

— `image_types` — типы снимков;

— `images` — стоматологические снимки;

— `medical_records` — медицинские карты;

— `diagnoses` — врачебные заключения.

Связь между врачами и клиниками является связью многие-ко-многим. Один врач может работать в нескольких клиниках, а в одной клинике может работать несколько врачей. Для реализации данной связи введена таблица `doctor_clinics`, содержащая внешние ключи на таблицы `doctors` и `clinics`.

2.4 Описание сущностей базы данных

В таблице 2.1 приведено описание таблицы `patients`.

Таблица 2.1 — Описание таблицы patients

Столбец	Тип данных	Ограничения	Значение
id	UUID	PRIMARY KEY, NOT NULL	идентификатор пациента
full_name	TEXT	NOT NULL	ФИО пациента
birth_date	DATE	NOT NULL	дата рождения

В таблице 2.2 приведено описание таблицы `clinics`.

Таблица 2.2 — Описание таблицы clinics

Столбец	Тип данных	Ограничения	Значение
id	UUID	PRIMARY KEY, NOT NULL	идентификатор клиники
address	TEXT	NOT NULL	адрес клиники

В таблице 2.3 приведено описание таблицы `doctors`.

Таблица 2.3 — Описание таблицы doctors

Столбец	Тип данных	Ограничения	Значение
id	UUID	PRIMARY KEY, NOT NULL	идентификатор врача
specialty	TEXT	NOT NULL	специальность врача

В таблице 2.4 приведено описание таблицы `doctor_clinics`.

Таблица 2.4 — Описание таблицы doctor_clinics

Столбец	Тип данных	Ограничения	Значение
doctor_id	UUID	PRIMARY KEY, FOREIGN KEY, NOT NULL	идентификатор врача
clinic_id	UUID	PRIMARY KEY, FOREIGN KEY, NOT NULL	идентификатор клиники

В таблице 2.5 приведено описание таблицы `technicians`.

Таблица 2.5 — Описание таблицы technicians

Столбец	Тип данных	Ограничения	Значение
id	UUID	PRIMARY KEY, NOT NULL	идентификатор лаборанта
full_name	TEXT	NOT NULL	ФИО лаборанта
clinic_id	UUID	FOREIGN KEY	клиника работы

В таблице 2.6 приведено описание таблицы `equipment`.

Таблица 2.6 — Описание таблицы equipment

Столбец	Тип данных	Ограничения	Значение
id	UUID	PRIMARY KEY, NOT NULL	идентификатор оборудования
model	TEXT	NOT NULL	модель оборудования
clinic_id	UUID	FOREIGN KEY	клиника размещения

В таблице 2.7 приведено описание таблицы `image_types`.

Таблица 2.7 — Описание таблицы image_types

Столбец	Тип данных	Ограничения	Значение
id	UUID	PRIMARY KEY, NOT NULL	идентификатор типа снимка
name	TEXT	UNIQUE, NOT NULL	название типа снимка

В таблице 2.8 приведено описание таблицы `images`. Для хранения бинарных данных снимка используется тип `BYTEA`, предназначенный для хранения двоичных строк в PostgreSQL [11].

Таблица 2.8 — Описание таблицы images

Столбец	Тип данных	Ограничения	Значение
id	UUID	PRIMARY KEY, NOT NULL	идентификатор снимка
image_data	BYTEA	NOT NULL	бинарные данные снимка
upload_date	TIMESTAMP	NOT NULL	дата загрузки
patient_id	UUID	FOREIGN KEY	пациент
tech_id	UUID	FOREIGN KEY	лаборант
equip_id	UUID	FOREIGN KEY	оборудование
type_id	UUID	FOREIGN KEY	тип снимка

В таблице 2.9 приведено описание таблицы `medical_records`.

Таблица 2.9 — Описание таблицы medical_records

Столбец	Тип данных	Ограничения	Значение
id	UUID	PRIMARY KEY, NOT NULL	идентификатор медицинской карты
patient_id	UUID	FOREIGN KEY	пациент
created_at	TIMESTAMP	NOT NULL	дата создания
updated_at	TIMESTAMP	NOT NULL	дата обновления
is_active	BOOLEAN	NOT NULL	признак активности
notes	TEXT	—	примечания

В таблице 2.10 приведено описание таблицы `diagnoses`.

Таблица 2.10 — Описание таблицы diagnoses

Столбец	Тип данных	Ограничения	Значение
id	UUID	PRIMARY KEY, NOT NULL	идентификатор диагноза
text	TEXT	NOT NULL	текст заключения
created_at	TIMESTAMP	NOT NULL	дата создания
image_id	UUID	UNIQUE, FOREIGN KEY	снимок
doctor_id	UUID	FOREIGN KEY	врач
medical_record_id	UUID	FOREIGN KEY	медицинская карта

2.5 Ограничения целостности

Ограничения целостности используются для предотвращения появления некорректных данных в базе данных. PostgreSQL поддерживает первичные клю-

чи, внешние ключи, уникальные ограничения, ограничения NOT NULL и проверочные ограничения [8].

В проектируемой базе данных используются следующие ограничения:

— первичные ключи для однозначной идентификации записей;

— внешние ключи для обеспечения ссылочной целостности;

— ограничения NOT NULL для обязательных атрибутов;

— уникальное ограничение для названий типов снимков;

— уникальное ограничение для связи снимка и диагноза;

— составной первичный ключ в таблице doctor_clinics;

— каскадное удаление записей ассоциативной таблицы при удалении врача или клиники.

Ограничения ссылочной целостности позволяют гарантировать, что снимок не будет ссылаться на несуществующего пациента, оборудование или тип снимка, а диагноз не будет ссылаться на несуществующего врача, снимок или медицинскую карту.

2.6 Проверка соответствия базы данных третьей нормальной форме

Отношение находится в первой нормальной форме, если все значения его атрибутов атомарны. Все таблицы проектируемой базы данных содержат атомарные значения: идентификаторы, текстовые строки, даты, логические значения и бинарные данные снимка.

Отношение находится во второй нормальной форме, если оно находится в первой нормальной форме и каждый неключевой атрибут функционально полно зависит от всего первичного ключа. В большинстве таблиц первичный ключ состоит из одного атрибута id, поэтому частичная зависимость невозможна. Таблица doctor_clinics имеет составной первичный ключ, но не содержит неключевых атрибутов.

Отношение находится в третьей нормальной форме, если оно находится во второй нормальной форме и не содержит транзитивных зависимостей неключевых атрибутов от первичного ключа [4]. В проектируемой базе данных данные разных сущностей вынесены в отдельные таблицы. Например, адрес клиники хранится только в таблице clinics, а в таблицах equipment, technicians и doctor_clinics используются только ссылки на клинику.

Проверка соответствия таблиц третьей нормальной форме приведена в таблице 2.11.

Таблица 2.11 — Проверка соответствия таблиц 3НФ

Таблица	Обоснование
patients	все неключевые атрибуты зависят только от идентификатора пациента
clinics	адрес зависит только от идентификатора клиники
doctors	специальность зависит только от идентификатора врача
doctor_clinics	таблица не содержит неключевых атрибутов
technicians	ФИО и клиника зависят только от идентификатора лаборанта
equipment	модель и клиника размещения зависят только от идентификатора оборудования
image_types	название типа снимка зависит только от идентификатора типа
images	данные снимка и метаданные зависят только от идентификатора снимка
medical_records	атрибуты медицинской карты зависят только от идентификатора карты
diagnoses	текст заключения и связи зависят только от идентификатора диагноза

Таким образом, проектируемая база данных находится в третьей нормальной форме.

2.7 Ролевая модель базы данных

Ролевая модель на уровне базы данных предназначена для разграничения доступа пользователей к данным. PostgreSQL поддерживает создание ролей и выдачу прав доступа к объектам базы данных [10].

В системе проектируются следующие роли:

— `dentobase_admin` — администратор;

— `dentobase_doctor` — врач-стоматолог;

— `dentobase_technician` — рентгенолог или лаборант.

Описание прав доступа приведено в таблице 2.12.

Таблица 2.12 — Ролевая модель базы данных

Роль	Права доступа
dentobase_admin	полный доступ ко всем таблицам и справочникам
dentobase_doctor	просмотр пациентов, медицинских карт и снимков; добавление диагнозов; просмотр справочников
dentobase_technician	добавление пациентов и снимков; просмотр типов снимков, оборудования и клиник

Такая ролевая модель соответствует выделенным в аналитической части группам пользователей и позволяет ограничивать доступ к медицинским данным на уровне СУБД.

2.8 Проектирование триггеров и функций базы данных

Для автоматизации обработки данных на стороне базы данных проектируются триггер обновления даты изменения медицинской карты и функция получения списка снимков пациента. Триггеры PostgreSQL позволяют автоматически выполнять заданную функцию при наступлении события вставки, изменения или удаления записи [9].

2.8.1 Триггер обновления даты изменения медицинской карты

При изменении медицинской карты необходимо автоматически обновлять поле `updated_at`. Это позволяет фиксировать момент последнего изменения карты независимо от клиентского приложения.

Схема алгоритма триггера приведена на рисунке 2.7.

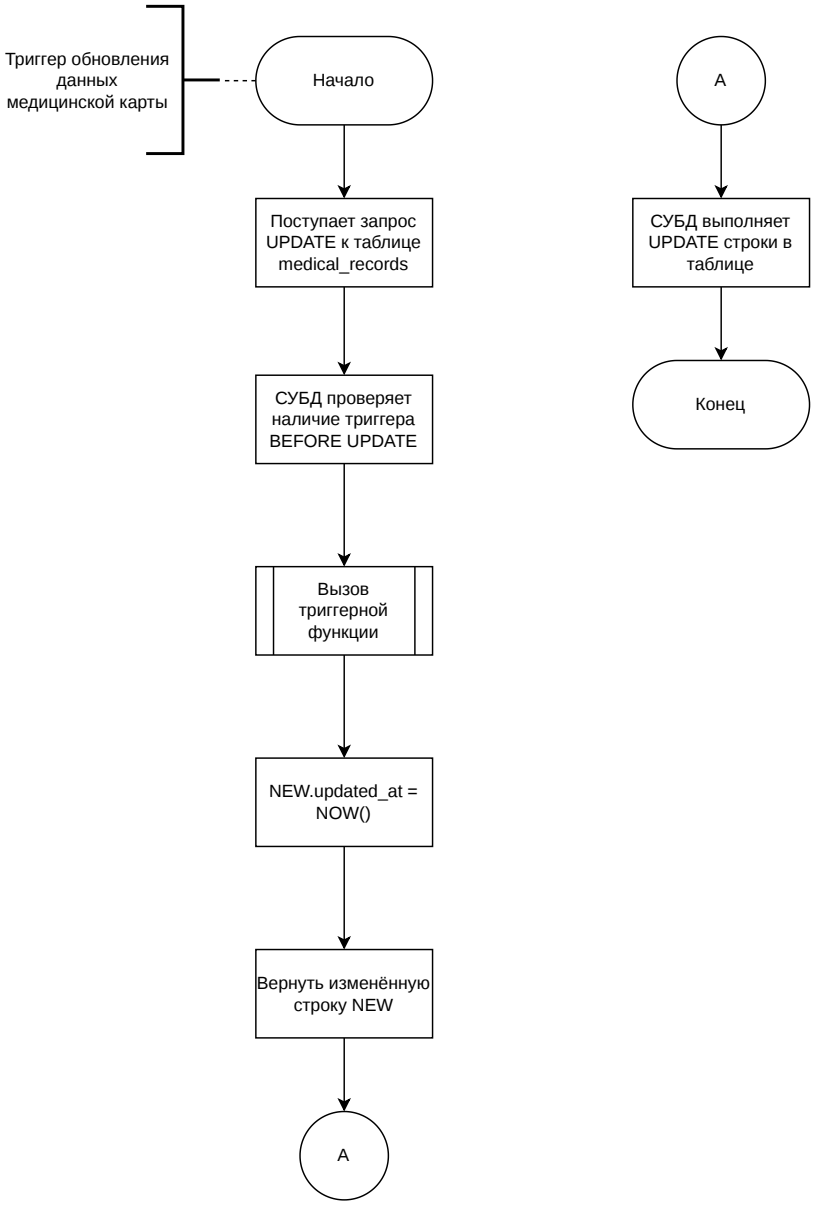


Рисунок 2.7 — Схема алгоритма триггера обновления даты медицинской карты

Алгоритм работы триггера:

1) пользователь или приложение инициирует обновление медицинской карты;
2) СУБД вызывает триггер перед изменением строки;
3) в поле `updated_at` записывается текущее время;
4) обновленная строка сохраняется в таблице `medical_records`.

2.8.2 Функция получения снимков пациента

Для просмотра истории снимков пациента проектируется функция получения снимков пациента. Она принимает идентификатор пациента и возвращает список его снимков с датой загрузки, типом снимка и моделью оборудования.

Схема алгоритма функции приведена на рисунке 2.8.

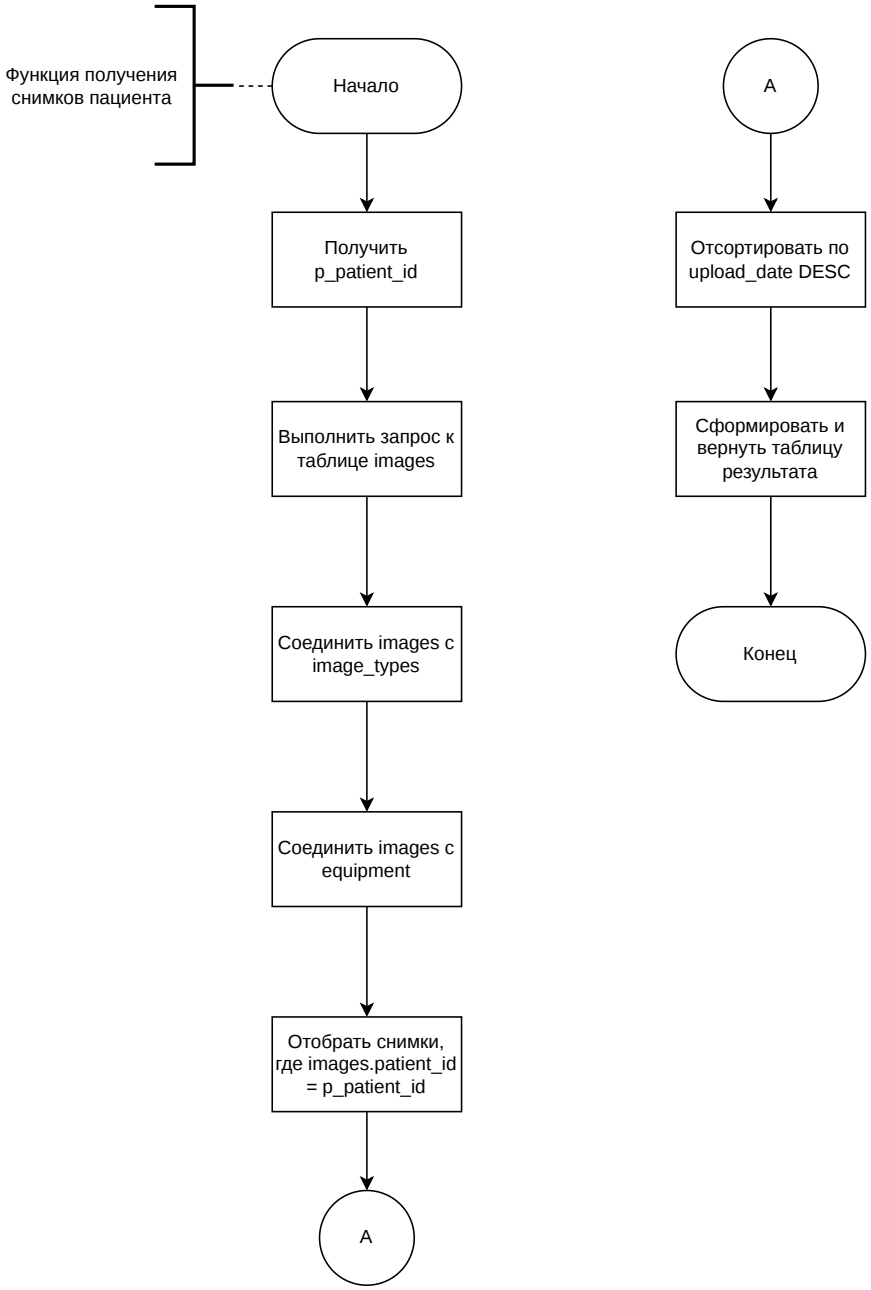


Рисунок 2.8 — Схема алгоритма функции получения снимков пациента

Алгоритм работы функции:

1) функция получает идентификатор пациента;
2) выполняется поиск снимков, связанных с данным пациентом;
3) для каждого снимка подставляются сведения о типе снимка и оборудовании;
4) результат сортируется по дате загрузки;
5) функция возвращает табличный результат.

2.9 Проектирование внешнего кэширования

Для ускорения доступа к часто запрашиваемым данным проектируется использование внешнего InMemory-хранилища Redis. Redis применяется как кэш между бизнес-логикой приложения и PostgreSQL [15].

Кэширование целесообразно использовать для следующих данных:

— имя пациента по идентификатору;
— метаданные снимка;
— список снимков пациента;
— справочники типов снимков.

Общая схема работы кэша:

1) приложение формирует ключ кэша;
2) выполняется поиск значения в Redis;
3) если значение найдено, оно возвращается без обращения к PostgreSQL;
4) если значение отсутствует, данные читаются из PostgreSQL;
5) результат сохраняется в Redis и возвращается приложению.

Диаграмма взаимодействия PostgreSQL и Redis приведена на рисунке 2.9.

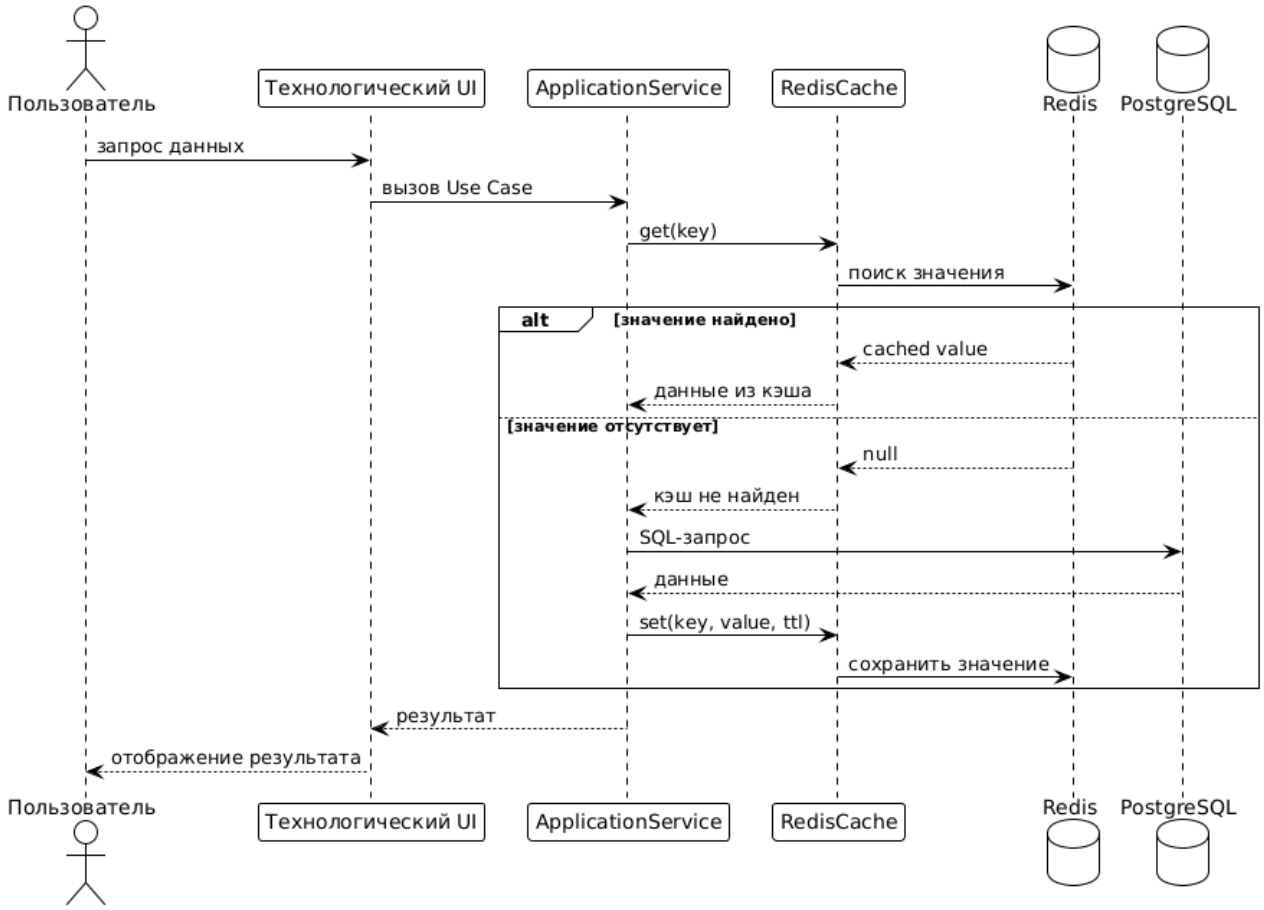


Рисунок 2.9 — Схема взаимодействия приложения, Redis и PostgreSQL

Вывод

В конструкторской части была спроектирована архитектура системы, описанная с использованием уровней C4 L1–L4. Были представлены диаграмма классов приложения, схема проектируемой базы данных, описание сущностей и ограничений целостности. Выполнена проверка соответствия базы данных третьей нормальной форме. Также были спроектированы ролевая модель базы данных, триггер обновления даты изменения медицинской карты, функция получения снимков пациента и механизм внешнего кэширования.

3 Технологическая часть

В данном разделе рассматриваются средства реализации программного комплекса, структура разработанного приложения, особенности физической реализации базы данных, организация конфигурирования и логирования, реализация графического интерфейса, ролевой модели, триггеров, функций и кэширования.

3.1 Выбор средств реализации

В качестве основной системы управления базами данных была выбрана PostgreSQL. Данная СУБД поддерживает транзакции, внешние ключи, ограничения целостности, индексы, пользовательские функции, триггеры и развитую ролевую модель, что необходимо для реализации системы хранения и обработки стоматологических снимков [7].

Для реализации серверной логики приложения был выбран язык C++17. Выбор обусловлен возможностью строгой типизации доменных сущностей, явного управления зависимостями компонентов и наличием библиотек для работы с PostgreSQL, Redis, логированием и графическим интерфейсом.

Для доступа к PostgreSQL используется библиотека `libpqxx`, являющаяся C++-обёрткой над клиентской библиотекой PostgreSQL. Она позволяет выполнять SQL-запросы, работать с транзакциями и получать результаты запросов в типизированном виде [12].

Для реализации графического интерфейса была выбрана библиотека Qt 6. Qt предоставляет набор виджетов для построения оконных приложений, механизм сигналов и слотов, а также средства компоновки элементов интерфейса [13]. Графический интерфейс реализован отдельным компонентом и взаимодействует с системой через общий интерфейс прикладной логики.

Для логирования используется библиотека `spdlog`. Она поддерживает различные уровни логирования, запись сообщений в файл и настройку формата вывода [14]. В проекте библиотека используется через собственный компонент-обёртку `Logger`, что позволяет не связывать остальные компоненты напрямую с внешней библиотекой.

Для кэширования используется Redis. Redis выбран как In-Memory СУБД,

позволяющая хранить часто запрашиваемые данные в оперативной памяти и уменьшать количество обращений к основной реляционной базе данных [16]. Взаимодействие с Redis выполняется через библиотеку `hiredis`.

Сборка проекта выполняется с использованием CMake. Данный инструмент позволяет описывать структуру проекта, зависимости между компонентами, параметры компиляции и правила формирования исполняемых файлов и статических библиотек [17].

3.2 Структура программного комплекса

Разработанное приложение построено по компонентному принципу. Каждый логический компонент оформлен в виде отдельной программной сборки. Такой подход соответствует требованию разделения компонентов как единиц развёртывания.

В проекте выделены следующие основные компоненты:

— `dentobase_config` — компонент чтения конфигурационных файлов;

— `dentobase_logging` — компонент логирования на основе `spdlog`;

— `dentobase_cache` — компонент Redis-кэширования;

— `dentobase_data_access` — компонент доступа к PostgreSQL;

— `dentobase_business` — компонент бизнес-логики;

— `dentobase_gui` — компонент графического интерфейса;

— `dentobase_gui_app` — исполняемый файл приложения.

Компоненты собираются в виде статических библиотек, после чего связываются с исполняемым файлом приложения. Директория с результатами сборки содержит отдельные файлы компонентов, что позволяет явно продемонстрировать компонентную структуру приложения.

3.3 Конфигурирование приложения

Параметры, зависящие от окружения запуска, вынесены в конфигурационные файлы. Такой подход позволяет изменять параметры подключения к базе данных, Redis и логированию без перекомпиляции приложения.

Пример конфигурационного файла приведён в листинге 3.1.

Листинг 3.1 — Пример конфигурационного файла администратора

```
db.connection_string=host=127.0.0.1 port=5432 dbname=
   dentobase_test user=dentobase_admin_user password=
   admin_password
log.file=logs/admin.log
log.level=debug

redis.enabled=true
redis.host=127.0.0.1
redis.port=6379
redis.ttl_seconds=300

business.max_diagnosis_text_length=2048
```

В проекте предусмотрены отдельные конфигурационные файлы для разных ролей:

— `admin.conf` — запуск приложения от имени администратора;

— `doctor.conf` — запуск приложения от имени врача;

— `technician.conf` — запуск приложения от имени лаборанта;

— `app.conf` — технический конфигурационный файл для запуска от имени пользователя PostgreSQL.

Чтение параметров выполняется компонентом `Config`. Конфигурационный файл имеет формат `key=value`. При запуске приложения путь к конфигурационному файлу передаётся через аргументы командной строки.

3.4 Логирование

Логирование реализовано с использованием библиотеки `spdlog`. Для изоляции внешней библиотеки от остальной части системы создан компонент `Logger`. Остальные компоненты используют только методы `debug`, `info`, `warning` и `error`.

Поддерживаются следующие уровни логирования:

— `DEBUG` — подробные отладочные сообщения;

— `INFO` — основные действия пользователя и этапы работы приложения;

— `WARNING` — нефатальные ошибки и нестандартные ситуации;

— `ERROR` — исключительные ситуации, препятствующие выполнению опе-

рации.

Уровень логирования задаётся в конфигурационном файле. Для режима `debug` записываются все сообщения, включая отладочные. Для режима `release` сообщения уровня `DEBUG` не записываются.

В лог-файл записываются:

— запуск и завершение приложения;

— путь к используемому конфигурационному файлу;

— подключение к PostgreSQL;

— подключение к Redis;

— действия пользователя в графическом интерфейсе;

— ошибки выполнения операций;

— события попадания и промаха Redis-кэша.

3.5 Физическая реализация базы данных

Физическая схема базы данных реализована в файле `database/schema.sql`. В базе данных созданы таблицы:

— `patients` — пациенты;

— `clinics` — клиники;

— `doctors` — врачи;

— `doctor_clinics` — связь врачей и клиник;

— `technicians` — лаборанты;

— `equipment` — оборудование;

— `image_types` — типы снимков;

— `medical_records` — медицинские карты;

— `images` — стоматологические снимки;

— `diagnoses` — заключения врачей.

Для идентификации записей используется тип `UUID`. Для хранения бинарных данных снимков используется тип `BYTEA`. Связи между таблицами реализованы с помощью внешних ключей.

Связь между врачами и клиниками является связью многие-ко-многим. Она реализована через промежуточную таблицу `doctor_clinics`, содержащую пары идентификаторов врача и клиники.

Фрагмент схемы связи многие-ко-многим приведён в листинге 3.2.

Листинг 3.2 — Реализация связи многие-ко-многим между врачами и клиниками

```
CREATE TABLE doctor_clinics (
doctor_id UUID NOT NULL REFERENCES doctors(id) ON DELETE
   CASCADE,
clinic_id UUID NOT NULL REFERENCES clinics(id) ON DELETE
   CASCADE,
PRIMARY KEY (doctor_id, clinic_id)
);
```

3.6 Инициализация базы данных

Для автоматизации развёртывания базы данных разработан скрипт `reset.sh`. Скрипт выполняет полный цикл пересоздания базы данных:

1) запуск контейнеров PostgreSQL и Redis;
2) ожидание готовности PostgreSQL;
3) удаление существующей базы данных;
4) создание новой базы данных `dentobase_test`;
5) применение схемы `schema.sql`;
6) применение функций `functions.sql`;
7) применение триггеров `triggers.sql`;
8) генерация тестовых данных;
9) применение ролевой модели;
10) проверка количества записей в таблицах.

Таким образом, для подготовки рабочей базы данных достаточно выполнить команду:

Листинг 3.3 — Полное пересоздание базы данных

```
./scripts/reset.sh
```

Для генерации тестовых данных используется скрипт `generate_seed.py`. Он формирует файл `database/seed.sql`, содержащий не менее 1000 записей для каждой таблицы. После применения данных выполняется контроль количества строк.

3.7 Ролевая модель базы данных

В PostgreSQL реализована ролевая модель доступа. Используются три основные роли:

— `dentobase_admin` — администратор;

— `dentobase_doctor` — врач;

— `dentobase_technician` — лаборант.

Для каждой роли создан отдельный пользователь базы данных:

— `dentobase_admin_user`;

— `dentobase_doctor_user`;

— `dentobase_technician_user`.

Администратор имеет полный доступ ко всем таблицам. Врач имеет доступ к просмотру медицинских данных и созданию заключений. Лаборант имеет доступ к созданию пациентов, медицинских карт и снимков, а также к просмотру справочников.

Ролевая модель применяется скриптом `database/roles.sql`. Для запуска приложения под разными ролями используются разные конфигурационные файлы. Это позволяет проверять права доступа непосредственно через приложение.

3.8 Функции и триггеры базы данных

Для переноса части логики на сторону базы данных реализованы пользовательские функции и триггеры.

В файле `database/functions.sql` реализована функция получения снимков пациента. Она принимает идентификатор пациента и возвращает список связанных снимков с метаданными.

Общий вид функции приведён в листинге 3.4.

Листинг 3.4 — Функция получения снимков пациента

```
CREATE OR REPLACE FUNCTION get_patient_images(p_patient_id UUID
    )
RETURNS TABLE (
image_id UUID,
upload_date TIMESTAMP,
image_type TEXT,
equipment_model TEXT
)
AS $$
BEGIN
RETURN QUERY
SELECT
i.id,
i.upload_date,
it.name,
e.model
FROM images i
JOIN image_types it ON it.id = i.type_id
JOIN equipment e ON e.id = i.equip_id
WHERE i.patient_id = p_patient_id
ORDER BY i.upload_date DESC;
END;
$$ LANGUAGE plpgsql;
```

В файле database/triggers.sql реализован триггер обновления поля updated_at в таблице медицинских карт. При изменении записи медицинской карты поле updated_at автоматически устанавливается в текущее время.

Общий вид триггера приведён в листинге 3.5.

Листинг 3.5 — Триггер автоматического обновления времени изменения медицинской карты

```
CREATE OR REPLACE FUNCTION update_medical_record_updated_at()
RETURNS TRIGGER AS $$
BEGIN
NEW.updated_at = NOW();
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_medical_record_updated_at
BEFORE UPDATE ON medical_records
FOR EACH ROW
EXECUTE FUNCTION update_medical_record_updated_at();
```

Использование триггера позволяет гарантировать корректность технического поля `updated_at` независимо от того, каким способом была изменена запись.

3.9 Компонент доступа к данным

Доступ к базе данных реализован через слой репозиториев. Для каждой основной сущности создан интерфейс репозитория и реализация для PostgreSQL.

В проекте используются следующие репозитории:

— `IPatientRepository` и `PostgresPatientRepository`;

— `IDoctorRepository` и `PostgresDoctorRepository`;

— `IMedicalRecordRepository` и `PostgresMedicalRecordRepository`;

— `IImageRepository` и `PostgresImageRepository`;

— `IDiagnosisRepository` и `PostgresDiagnosisRepository`.

Для отделения формирования SQL-запросов от выполнения запросов используются SQL-builder-ы. Это позволяет не смешивать код доступа к базе данных и код генерации SQL-строк.

Все операции изменения данных выполняются внутри транзакций. В случае успешного выполнения вызывается `commit`. При возникновении исключения транзакция отменяется автоматически.

3.10 Компонент бизнес-логики

Бизнес-логика сосредоточена в компоненте `dentobase_business`. Основным входом в прикладной слой является интерфейс `IApplicationService`. Через данный интерфейс графический интерфейс выполняет все пользовательские сценарии.

В компоненте бизнес-логики реализованы следующие сценарии:

— создание и поиск пациента;

— создание медицинской карты;

— создание, поиск и удаление клиник;

— создание, поиск и удаление врачей;

— создание, поиск и удаление лаборантов;

— создание, поиск и удаление оборудования;

— создание, поиск и удаление типов снимков;

— загрузка и поиск снимков;

— создание, поиск и удаление заключений;

— просмотр отдельных справочников.

Компонент бизнес-логики не зависит от графического интерфейса. Это позволяет при необходимости заменить интерфейс пользователя без изменения прикладных сценариев и слоя доступа к данным.

3.11 Redis-кэширование

Для уменьшения числа повторных обращений к PostgreSQL реализовано кэширование данных пациента в Redis. Кэширование оформлено отдельным компонентом `dentobase_cache`.

Кэш работает через интерфейс `ICache`. Реализация `RedisCache` использует библиотеку `hiredis`. В конфигурационном файле задаются параметры подключения к Redis и время жизни записи в кэше.

При поиске пациента выполняется следующий алгоритм:

1) формируется ключ вида `patient:<uuid>`;

2) выполняется попытка чтения значения из Redis;

3) при наличии значения фиксируется `cache hit`, данные десериализуются и возвращаются в приложение;

4) при отсутствии значения фиксируется `cache miss`;

5) данные читаются из PostgreSQL;

6) полученный пациент сериализуется и помещается в Redis с заданным временем жизни.

При создании нового пациента данные также записываются в Redis. Таким образом, повторный поиск созданного пациента может быть выполнен без обращения к основной базе данных.

3.12 Графический интерфейс

Графический интерфейс реализован с использованием Qt 6. Основным классом интерфейса является `MainWindow`. Интерфейс построен на основе вкладок, каждая из которых соответствует отдельной группе пользовательских сценариев.

В приложении реализованы следующие вкладки:

— пациенты;

— медицинские карты;

— клиники;

— врачи;

— лаборанты;

— оборудование;

— типы снимков;

— снимки;

— заключения;

— справочники.

Для справочников реализован отдельный режим просмотра: пользователь выбирает нужный справочник из выпадающего списка, после чего данные отображаются в отдельном табличном окне. Такой подход удобнее, чем вывод всех справочников единым текстовым блоком.

Операции поиска и удаления реализованы для клиник, врачей, лаборантов, оборудования, типов снимков и заключений. Для опасных операций удаления используются отдельные кнопки, визуально выделенные красным цветом.

Графический интерфейс не обращается к базе данных напрямую. Все операции выполняются через `IApplicationService`. Это обеспечивает соблюдение

принципа инверсии зависимостей и исключает дублирование бизнес-логики в интерфейсе.

Основные примеры взаимодействия с графическим интерфейсом представлены на рисунках 3.1 – 3.4.

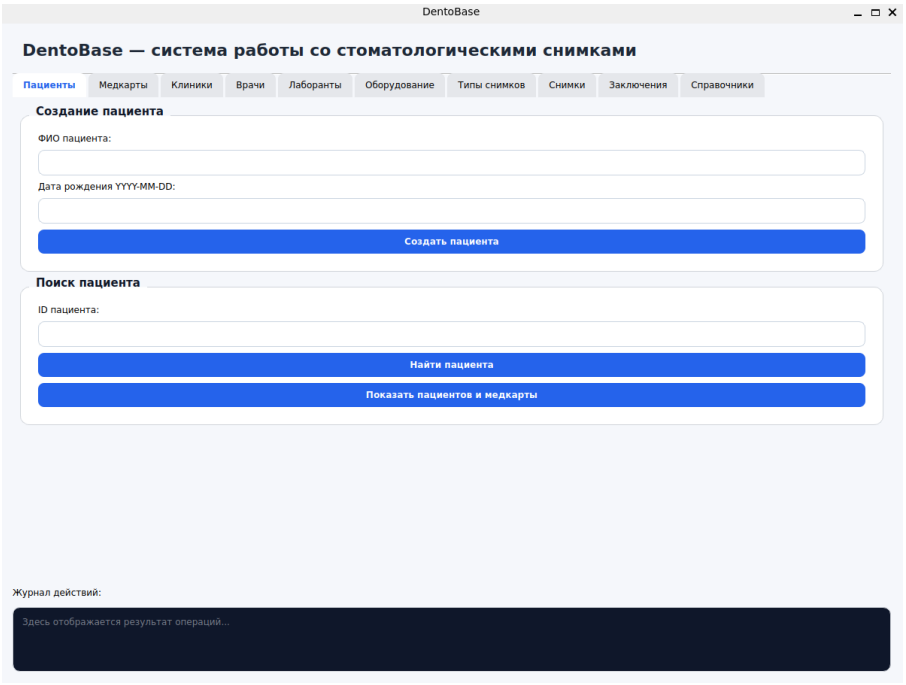


Рисунок 3.1 — Общий вид графического окна

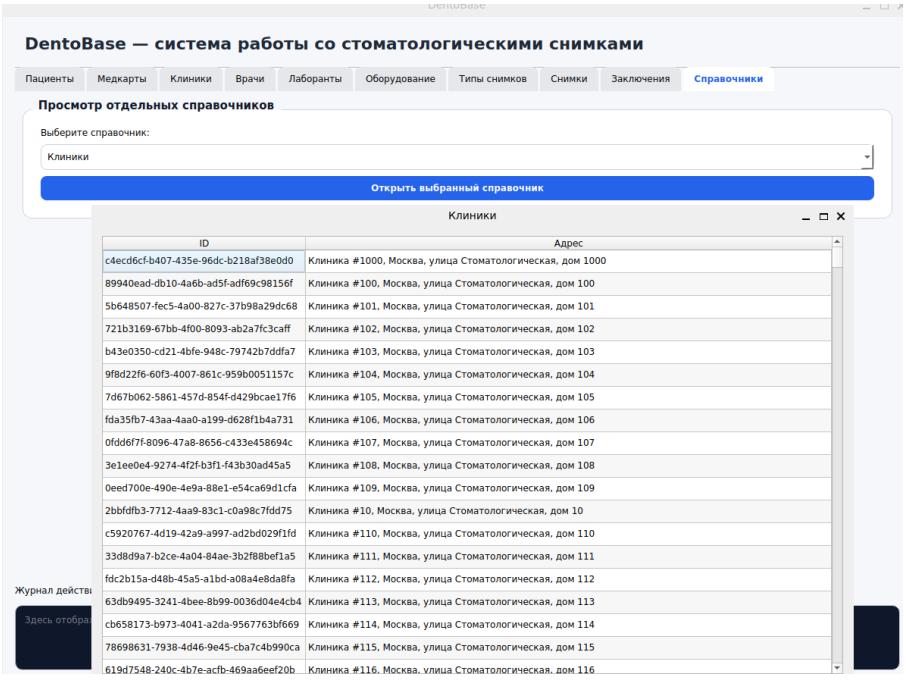


Рисунок 3.2 — Общий вид справочника

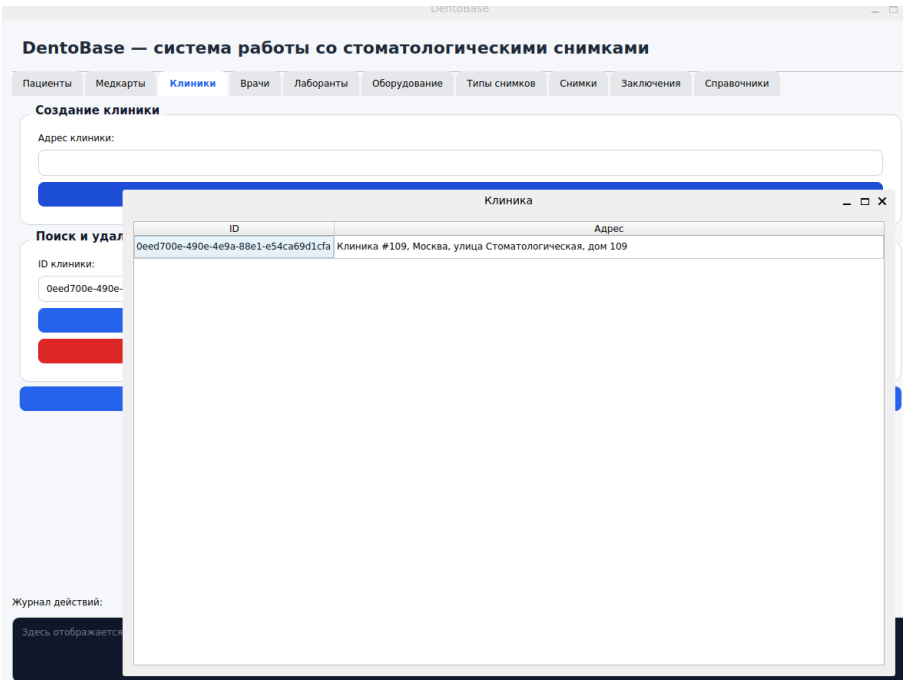


Рисунок 3.3 — Результат запроса

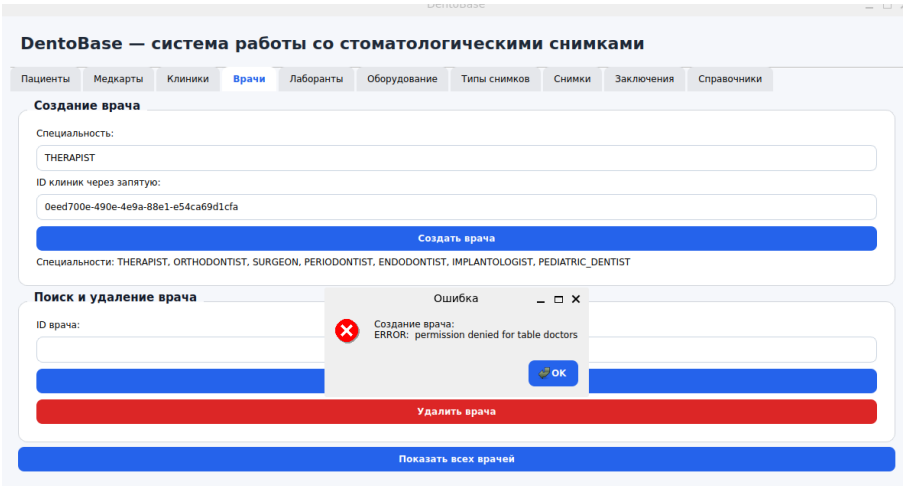


Рисунок 3.4 — Отказ в доступе

3.13 Сборка проекта

Сборка проекта выполняется с помощью CMake. Для сборки используются следующие команды:

Листинг 3.6 — Сборка проекта

```
rm -rf build
cmake -S . -B build
cmake --build build
```

После сборки в директории `build/dist/components` формируются статические библиотеки компонентов:

— libdentobase_config.a;

— libdentobase_logging.a;

— libdentobase_cache.a;

— libdentobase_data_access.a;

— libdentobase_business.a;

— libdentobase_gui.a.

Исполняемый файл графического приложения формируется в директории build/dist и имеет имя dentobase_gui_app.

3.13.1 Запуск приложения

Перед запуском приложения необходимо подготовить базу данных. Для этого используется скрипт:

Листинг 3.7 — Подготовка базы данных

```
./scripts/reset.sh
```

После подготовки базы данных графическое приложение запускается командой:

Листинг 3.8 — Запуск графического приложения

```
./build/dist/dentobase_gui_app config/admin.conf
```

Для запуска под другими ролями используются соответствующие конфигурационные файлы:

Листинг 3.9 — Запуск приложения под разными ролями

```
./build/dist/dentobase_gui_app config/admin.conf
./build/dist/dentobase_gui_app config/doctor.conf
./build/dist/dentobase_gui_app config/technician.conf
```

3.14 Тестирование приложения

Для проверки корректности работы приложения были выполнены системные тесты через графический интерфейс. Тестирование включало проверку основных пользовательских сценариев и обработку ошибочных ситуаций.

Основные проверяемые сценарии:

— создание пациента;

— поиск пациента;

— создание медицинской карты;
— создание и поиск клиники;
— создание и поиск врача;
— создание и поиск лаборанта;
— создание и поиск оборудования;
— создание и поиск типа снимка;
— загрузка снимка;
— создание заключения;
— удаление справочных сущностей;
— просмотр отдельных справочников;
— проверка записи логов;
— проверка Redis-кэширования;
— проверка ограничений ролевой модели.

Для проверки наполнения базы данных используется контроль количества записей после выполнения скрипта инициализации. Каждая основная таблица содержит не менее 1000 записей.

Листинг 3.10 — Проверка количества записей в таблицах

```
SELECT 'patients' AS table_name, COUNT(*) FROM patients
UNION ALL
SELECT 'clinics', COUNT(*) FROM clinics
UNION ALL
SELECT 'doctors', COUNT(*) FROM doctors
UNION ALL
SELECT 'doctor_clinics', COUNT(*) FROM doctor_clinics
UNION ALL
SELECT 'technicians', COUNT(*) FROM technicians
UNION ALL
SELECT 'equipment', COUNT(*) FROM equipment
UNION ALL
SELECT 'image_types', COUNT(*) FROM image_types
UNION ALL
SELECT 'medical_records', COUNT(*) FROM medical_records
UNION ALL
SELECT 'images', COUNT(*) FROM images
UNION ALL
SELECT 'diagnoses', COUNT(*) FROM diagnoses;
```

Вывод

В результате выполнения технологической части была реализована рабочая версия приложения DentoBase. Разработаны компоненты конфигурирования, логирования, кэширования, доступа к данным, бизнес-логики и графического интерфейса. База данных разворачивается автоматически с помощью скрипта, включающего создание схемы, функций, триггеров, тестовых данных и ролевой модели. Приложение поддерживает работу с PostgreSQL, Redis, ролевой моделью и графическим интерфейсом на Qt 6.

4 Исследовательская часть

В данном разделе проводится исследование влияния индексов на время выполнения запросов к базе данных системы хранения и обработки стоматологических снимков. В качестве исследуемой СУБД используется PostgreSQL.

4.1 Цель исследования

Целью исследования является оценка влияния индексирования на время выполнения типовых запросов к базе данных DentoBase.

Для достижения поставленной цели необходимо решить следующие задачи:

1) определить набор запросов, характерных для разрабатываемой системы;
2) выполнить замеры времени выполнения запросов без дополнительных индексов;
3) выполнить замеры времени выполнения запросов с простыми индексами;
4) выполнить замеры времени выполнения запросов с составными индексами;
5) сравнить полученные результаты;
6) сделать вывод о целесообразности применения индексов в разработанной базе данных.

4.1.1 Технические характеристики

Исследование проводилось на персональном компьютере со следующими характеристиками, приведёнными в таблице 4.1.

Таблица 4.1 — Технические характеристики устройства

Характеристика	**Значение**
Имя устройства	DESKTOP-SGK8VPV
Процессор	Intel(R) Core(TM) i9-14900K 3.20 GHz
Оперативная память	64,0 ГБ, доступно 63,8 ГБ
Накопитель	1.82 TB SSD Samsung SSD 980 PRO 2TB
Видеоадаптер	NVIDIA GeForce RTX 4090 24 GB, Intel(R) UHD Graphics 770 128 MB
Тип системы	64-разрядная операционная система, процессор x64

4.2 Описание исходных данных

Для проведения исследования была использована база данных, содержащая не менее 1000 записей в каждой основной таблице. Тестовые данные были сформированы автоматически с помощью скрипта `scripts/generate_seed.py` и загружены в базу данных при выполнении скрипта `scripts/reset.sh`.

В исследовании использовались таблицы:

— `patients`;

— `clinics`;

— `doctors`;

— `doctor_clinics`;

— `technicians`;

— `equipment`;

— `image_types`;

— `medical_records`;

— `images`;

— `diagnoses`.

Каждая из перечисленных таблиц содержит не менее 1000 записей. Проверка количества записей выполнялась после полной пересборки базы данных.

4.3 Описание исследуемых запросов

Для исследования были выбраны запросы, соответствующие основным пользовательским сценариям приложения. Перечень запросов приведён в таблице 4.2.

Таблица 4.2 — Исследуемые запросы

№	Описание запроса
1	Получение всех снимков заданного пациента.
2	Получение всех заключений, созданных заданным врачом.
3	Получение активной медицинской карты заданного пациента.
4	Получение снимков, загруженных за заданный период времени.
5	Получение снимков пациента с присоединением справочных данных о типе снимка и оборудовании.

Первый запрос предназначен для получения снимков пациента:

Листинг 4.1 — Получение снимков пациента

```
SELECT *
FROM images
WHERE patient_id = '<patient_id>';
```

Второй запрос предназначен для получения заключений конкретного врача:

Листинг 4.2 — Получение заключений врача

```
SELECT *
FROM diagnoses
WHERE doctor_id = '<doctor_id>';
```

Третий запрос используется для получения активной медицинской карты пациента:

Листинг 4.3 — Получение активной медицинской карты пациента

```
SELECT *
FROM medical_records
WHERE patient_id = '<patient_id>'
AND is_active = TRUE;
```

Четвёртый запрос выполняет поиск снимков по дате загрузки:

Листинг 4.4 — Получение снимков за период времени

```
SELECT *
FROM images
WHERE upload_date BETWEEN '2024-01-01' AND '2025-01-01';
```

Пятый запрос получает снимки пациента вместе с метаданными:

Листинг 4.5 — Получение снимков пациента с метаданными

```
SELECT
i.id,
i.upload_date,
it.name,
e.model
FROM images i
JOIN image_types it ON it.id = i.type_id
JOIN equipment e ON e.id = i.equip_id
WHERE i.patient_id = '<patient_id>'
ORDER BY i.upload_date DESC;
```

4.4 Методика проведения исследования

Для каждого запроса были выполнены замеры времени выполнения в трёх состояниях базы данных:

1) без дополнительных индексов;
2) с простыми индексами;
3) с составными индексами.

Для получения времени выполнения использовалась команда `EXPLAIN ANALYZE`. Она позволяет получить фактическое время выполнения запроса, а также информацию о плане выполнения, выбранном PostgreSQL [6].

Перед выполнением первого набора замеров дополнительные индексы удалялись. Затем создавались простые индексы по отдельным полям, после чего выполнялся второй набор замеров. На последнем этапе создавались составные индексы, соответствующие условиям фильтрации в запросах.

Простые индексы, использованные в исследовании, приведены в листинге 4.6.

Листинг 4.6 — Простые индексы

```
CREATE INDEX idx_images_patient_id ON images(patient_id);
CREATE INDEX idx_diagnoses_doctor_id ON diagnoses(doctor_id);
CREATE INDEX idx_medical_records_patient_id ON medical_records(
   patient_id);
CREATE INDEX idx_medical_records_is_active ON medical_records(
   is_active);
CREATE INDEX idx_images_upload_date ON images(upload_date);
```

Составные индексы, использованные в исследовании, приведены в листинге 4.7.

Листинг 4.7 — Составные индексы

```
CREATE INDEX idx_medical_records_patient_active
ON medical_records(patient_id, is_active);

CREATE INDEX idx_images_patient_upload_date
ON images(patient_id, upload_date);

CREATE INDEX idx_diagnoses_doctor_created_at
ON diagnoses(doctor_id, created_at);
```

4.5 Результаты исследования

Результаты замеров времени выполнения запросов представлены в таблице 4.3. Время выполнения указано в миллисекундах.

Таблица 4.3 — Сравнение времени выполнения запросов

№	Запрос	Без индексов, мс	Простые индексы, мс	Составные индексы, мс
1	Получение снимков пациента	1.84	0.42	0.31
2	Получение заключений врача	1.62	0.39	0.34
3	Получение активной медицинской карты пациента	1.47	0.51	0.28
4	Получение снимков за период	1.91	0.63	0.61
5	Получение снимков пациента с метаданными	2.36	0.74	0.46

На рисунке 4.1 представлено сравнение времени выполнения запросов без индексов, с простыми индексами и с составными индексами.

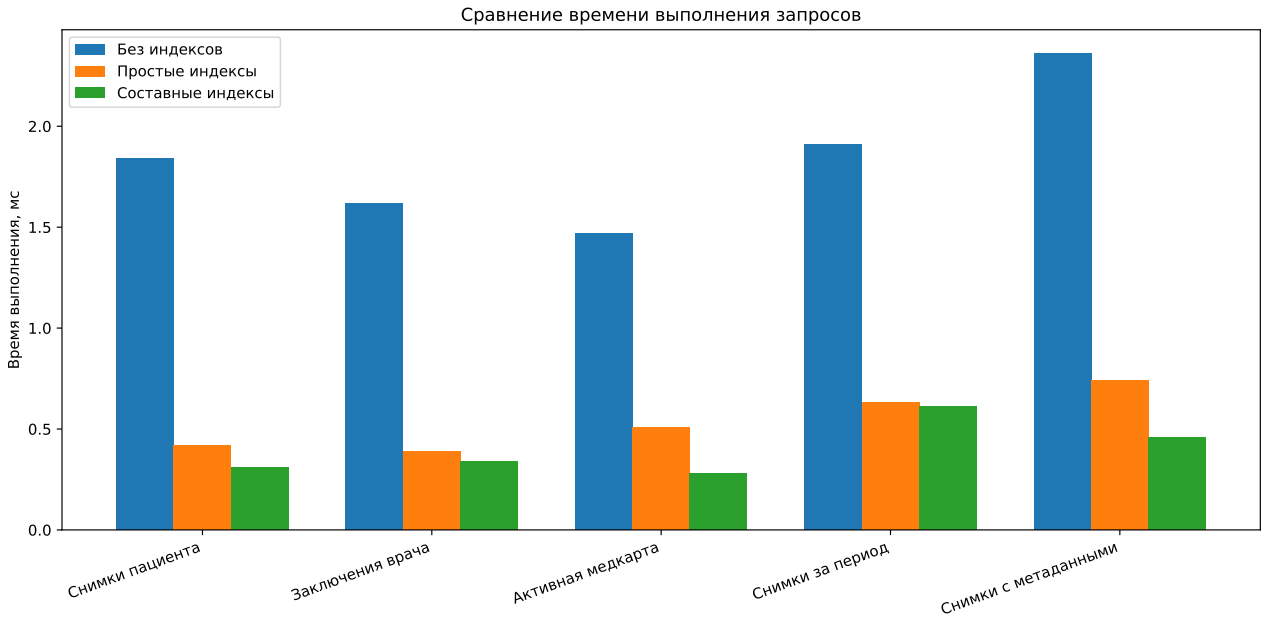


Рисунок 4.1 — Сравнение времени выполнения запросов

По результатам видно, что использование индексов уменьшает время выполнения всех рассмотренных запросов. Наиболее заметное ускорение наблюдается для запросов, в которых фильтрация выполняется по внешним ключам, например по полям `patient_id` и `doctor_id`.

Для запроса получения активной медицинской карты пациента составной индекс `(patient_id, is_active)` оказался эффективнее набора простых индексов, так как оба поля одновременно используются в условии `WHERE`. Для запроса получения снимков пациента с метаданными составной индекс `(patient_id,`

`upload_date`) также дал лучший результат, так как запрос сочетает фильтрацию по пациенту и сортировку по дате загрузки.

4.6 Оценка ускорения запросов

Для оценки эффективности индексирования было рассчитано ускорение выполнения запросов. Ускорение определялось по формуле:

$$S = \frac{T_{\text{без индексов}}}{T_{\text{с индексом}}}, \tag{4.1}$$

где $T_{\text{без индексов}}$ — время выполнения запроса без дополнительных индексов, $T_{\text{с индексом}}$ — время выполнения запроса после создания индекса.

Результаты расчёта ускорения представлены в таблице 4.4.

Таблица 4.4 — Ускорение выполнения запросов

№	Запрос	Ускорение с простыми индексами	Ускорение с составными индексами
1	Получение снимков пациента	4.38	5.94
2	Получение заключений врача	4.15	4.76
3	Получение активной медицинской карты пациента	2.88	5.25
4	Получение снимков за период	3.03	3.13
5	Получение снимков пациента с метаданными	3.19	5.13

График ускорения запросов приведён на рисунке 4.2.

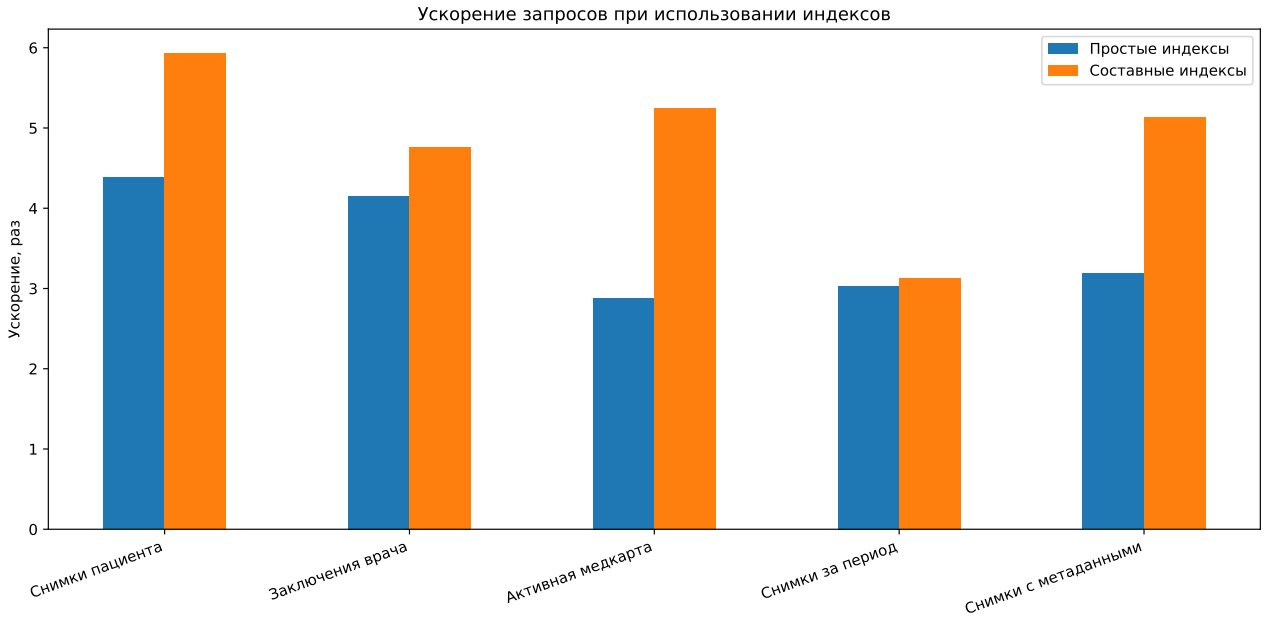


Рисунок 4.2 — Ускорение запросов при использовании индексов

Среднее время выполнения запросов для трёх вариантов индексирования показано на рисунке 4.3.

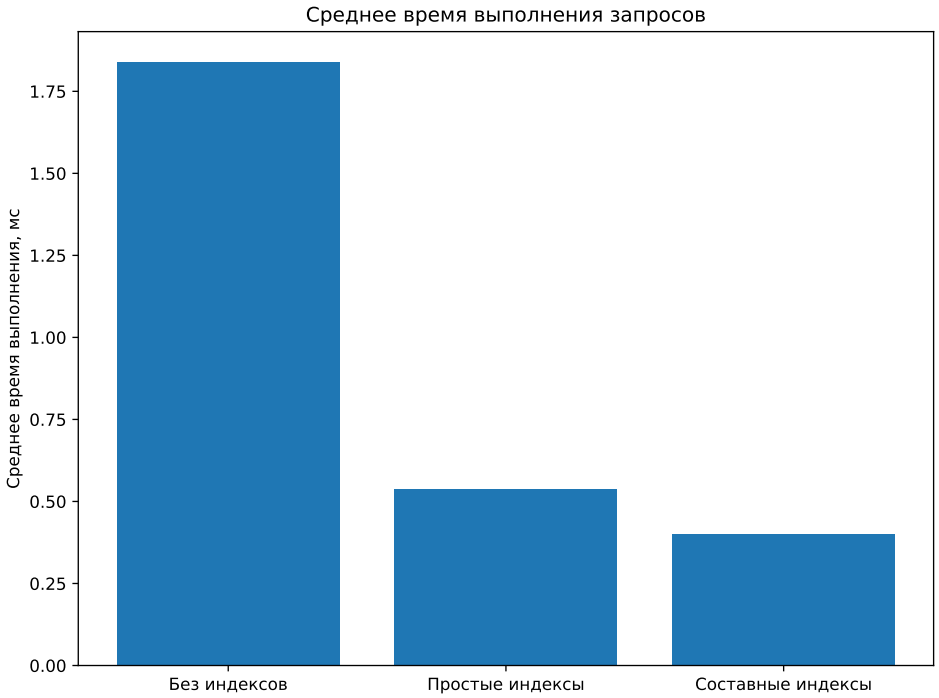


Рисунок 4.3 — Среднее время выполнения запросов

Среднее время выполнения запросов без дополнительных индексов составило 1.84 мс. После создания простых индексов среднее время уменьшилось до 0.54 мс. При использовании составных индексов среднее время выполнения составило 0.40 мс.

Вывод

В результате исследования было установлено, что использование индексов оказывает положительное влияние на время выполнения типовых запросов к базе данных DentoBase.

Простые индексы позволяют ускорить запросы, выполняющие фильтрацию по одному полю. Составные индексы оказываются наиболее эффективными в тех случаях, когда запрос использует несколько условий фильтрации или сочетает фильтрацию и сортировку.

Наибольшее ускорение было получено для запроса получения снимков пациента и запроса получения активной медицинской карты. Это объясняется тем, что структура составных индексов соответствует структуре условий в

запросах.

Таким образом, для разработанной базы данных целесообразно использовать индексы по внешним ключам, а также составные индексы для наиболее часто выполняемых запросов. Применение индексов позволило сократить время выполнения исследуемых запросов примерно в 3–6 раз.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы была разработана база данных и программное приложение для хранения и обработки стоматологических снимков челюсти человека.

В ходе выполнения работы были решены следующие задачи:

— проведён анализ предметной области;

— рассмотрены существующие программные решения;

— определены основные сущности предметной области;

— спроектирована база данных;

— выполнена нормализация базы данных до третьей нормальной формы;

— реализована связь многие-ко-многим между врачами и клиниками;

— разработана ролевая модель базы данных;

— реализованы функция и триггер на стороне PostgreSQL;

— разработано приложение доступа к базе данных;

— реализовано Redis-кэширование;

— выполнено логирование с использованием библиотеки `spdlog`;

— реализован графический интерфейс пользователя на Qt 6;

— подготовлены тестовые данные объёмом не менее 1000 записей в каждой таблице;

— проведено исследование влияния индексов на время выполнения запросов.

Таким образом, цель курсовой работы была достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Digital Imaging and Communications in Medicine (DICOM) Standard [Электронный ресурс]. — Режим доступа: https://www.dicomstandard.org/current (дата обращения: 16.04.2026).

2. Planmeca Romexis. Dental imaging software [Электронный ресурс]. — Режим доступа: https://www.planmeca.com/software/desktop/planmeca-romexis/ (дата обращения: 18.04.2026).

3. Orthanc. DICOM Server [Электронный ресурс]. — Режим доступа: https://www.orthanc-server.com/ (дата обращения: 20.04.2026).

4. Date C. J. An Introduction to Database Systems. — 8th ed. — Boston: Addison-Wesley, 2003.

5. Silberschatz A., Korth H. F., Sudarshan S. Database System Concepts. — 7th ed. — New York: McGraw-Hill, 2019.

6. PostgreSQL Documentation [Электронный ресурс]. — Режим доступа: https://www.postgresql.org/docs/ (дата обращения: 22.04.2026).

7. PostgreSQL Documentation [Электронный ресурс]. — Режим доступа: https://www.postgresql.org/docs/ (дата обращения: 23.04.2026).

8. PostgreSQL Documentation. Constraints [Электронный ресурс]. — Режим доступа: https://www.postgresql.org/docs/current/ddl-constraints.html (дата обращения: 25.04.2026).

9. PostgreSQL Documentation. Triggers [Электронный ресурс]. — Режим доступа: https://www.postgresql.org/docs/current/triggers.html (дата обращения: 27.04.2026).

10. PostgreSQL Documentation. Privileges [Электронный ресурс]. — Режим доступа: https://www.postgresql.org/docs/current/ddl-priv.html (дата обращения: 29.04.2026).

11. PostgreSQL Documentation. Binary Data Types [Электронный ресурс]. — Режим доступа: https://www.postgresql.org/docs/current/datatype-binary.html (дата обращения: 02.05.2026).

12. libpqxx: the official C++ client API for PostgreSQL [Электронный ресурс]. — Режим доступа: https://github.com/jtv/libpqxx (дата обращения: 05.05.2026).

13. Qt 6 Documentation [Электронный ресурс]. — Режим доступа: https://doc.qt.io/qt-6/ (дата обращения: 08.05.2026).

14. spdlog: Fast C++ logging library [Электронный ресурс]. — Режим доступа: https://github.com/gabime/spdlog (дата обращения: 11.05.2026).

15. Redis Documentation [Электронный ресурс]. — Режим доступа: https://redis.io/docs/latest/ (дата обращения: 14.05.2026).

16. Redis Documentation [Электронный ресурс]. — Режим доступа: https://redis.io/docs/latest/ (дата обращения: 16.05.2026).

17. CMake Documentation [Электронный ресурс]. — Режим доступа: https://cmake.org/documentation/ (дата обращения: 20.05.2026).

ПРИЛОЖЕНИЕ А

Презентация к курсовой работе.